

VOYAGER：具有大型语言模型的开放式嵌入式代理

Guanzhi Wang¹✉, Yuqi Xie³, Yunfan Jiang^{4*}, Ajay Mandlekar^{1*}, Chaowei Xiao⁽¹⁾⁵, Yuke Zhu⁽¹⁾³, Linxi "Jim" Fan⁽¹⁾⁽⁺⁾✉, Anima Anandkumar⁽¹⁾⁽²⁾⁽⁺⁾⁽¹⁾ 英伟达公司, ²加州理工学院,

³UT Austin, ⁽⁴⁾斯坦福大学, ⁵华盛顿大学麦迪逊分校

*贡献相同 †建议相同 ✉ 通讯作者

<https://voyager.minedojo.org>

摘要

我们介绍 VOYAGER，它是 Minecraft 中第一个由 LLM 驱动的嵌入式终身学习代理，可以在没有人类干预的情况下不断探索世界、获取各种技能并做出新发现。VOYAGER 由三个关键部分组成：1) 最大化探索的自动课程；2) 用于存储和检索复杂行为的不断增长的可执行代码技能库；3) 结合环境反馈、执行错误和自我验证以改进程序的全新迭代提示机制。VOYAGER 通过黑盒查询与 GPT-4 进行交互，从而避免了模型参数微调的需要。VOYAGER 开发的技能具有时间扩展性、可解释性和组合性，能迅速增强代理的能力，减少灾难性遗忘。经验表明，VOYAGER 具有很强的情境终身学习能力，在玩 Minecraft 时表现出了非凡的熟练程度。与之前的 SOTA 相比，VOYAGER 获得的独特物品多 3.3 $\times$ ，旅行距离远 2.3 $\times$ ，解锁关键技术树里程碑的速度快达 15.3 $\times$ 。VOYAGER 能够在新的 Minecraft 世界中利用学习到的技能库从头开始解决新任务，而其他技术则很难做到这一点。

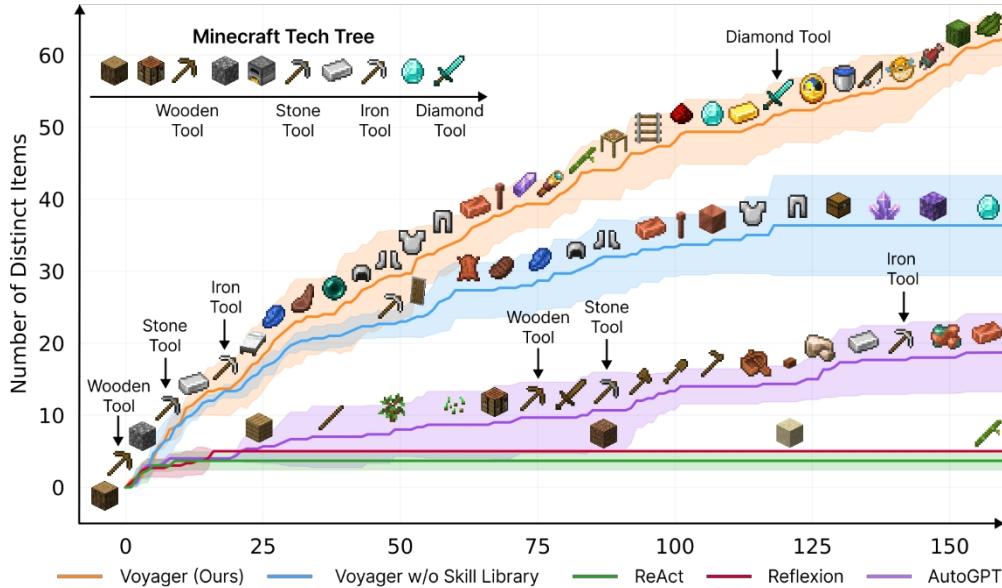


图 1：VOYAGER 通过自我驱动探索不断发现新的 Minecraft 物品和技能，性能明显优于基线。X 轴表示提示迭代次数。

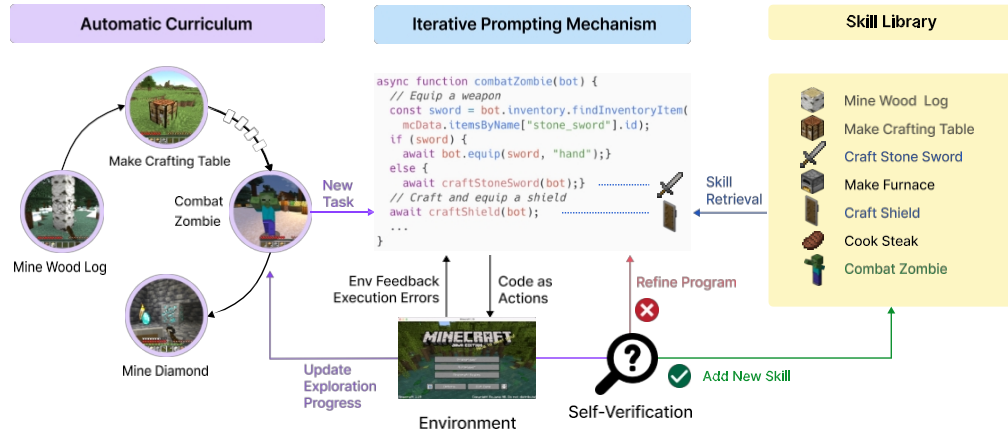


图 2：VOYAGER 由三个关键部分组成：用于开放式探索的自动课程、用于日益复杂行为的技能库，以及使用代码作为行动空间的迭代提示机制。

1 简介

人工智能界面临的一个巨大挑战是建立具有一般能力的、能在开放世界中不断探索、规划和发展新技能的具身代理[1-5]。经典方法采用强化学习（RL）[6, 7]和模仿学习[8-10]，对原始动作进行操作，这对系统探索[11-15]、可解释性[16-18]和泛化[19-21]都是挑战。最近，基于大型语言模型（LLM）的代理利用预先训练好的 LLM 中封装的世界知识来生成一致的行动计划或可执行策略[16, 22, 19]。它们被应用于游戏和机器人等具身任务 [23-27]，以及无具身的 NLP 任务 [28-30]。然而，这些代理并不是终身学习者，无法在较长的时间跨度内逐步获取、更新、积累和转移知识[31, 32]。

让我们以 Minecraft 为例。与人工智能领域研究的大多数其他游戏不同[33, 34, 10]，Minecraft 并不强加一个预定义的最终目标或固定的故事情节，而是提供了一个具有无限可能性的独特游乐场[23]。Minecraft 要求玩家探索广阔的、按程序生成的 3D 地形，并利用收集到的资源解锁技术树。人类玩家通常从学习采矿和烹饪食物等基础知识开始，然后再学习打击怪物和制作钻石工具等更复杂的任务。我们认为，一个有效的终身学习代理应该具备与人类玩家类似的能力：(1) 根据其当前的技能水平和世界状态**提出合适的任务**，例如，如果它发现自己身处沙漠而非森林，则应先学习收获沙子和仙人掌，然后再学习打铁；(2) 根据环境反馈**完善技能，并将掌握的技能保存在记忆中**，以便将来在类似情况下重复使用（例如，打僵尸与打蜘蛛类似）；(3) **不断探索世界**，并以自我驱动的方式寻找新任务。

为了实现这些目标，我们推出了 VOYAGER，它是第一个由 LLM 驱动的**嵌入式终身学习代理**，可以在 Minecraft 中驱动探索、掌握各种技能并不断做出新发现，而无需人工干预。VOYAGER 通过三个关键模块得以实现（图 2）：1）可最大化探索的**自动课程**；2）用于存储和检索复杂行为的**技能库**；3）用于生成可执行代码的新型**迭代提示机制**。我们选择使用代码作为行动空间，而不是低级运动指令，因为程序可以自然地表示时间扩展和组合行动[16, 22]，这对 Minecraft 中的许多长视距任务至关重要。VOYAGER 通过提示和上下文学习与黑盒 LLM（GPT-4 [35]）进行交互 [36-38]。我们的方法绕过了对模型参数访问和基于梯度的显式训练或微调的需求。

更具体地说，VOYAGER 尝试解决**自动课程**提出的难度逐渐增加的任务，该**课程**考虑了探索进度和代理的状态。课程由 GPT-4 根据“发现尽可能多的不同事物”这一总体目标生成。这种方法可以看作是一种情境中的**新奇搜索**形式 [39, 40]。VOYAGER 通过存储有助于成功完成任务的操作程序，逐步建立**技能库**。



图 3：自动课程提出的任务。为简洁起见，我们只显示了部分提示。完整的提示结构见附录 A.3 节。

每个程序都有其描述的嵌入索引，可在今后类似情况下进行检索。复杂的技能可以通过组合较简单的程序来合成，从而使 VOYAGER 的能力随着时间的推移迅速增强，并减轻了其他持续学习方法中的灾难性遗忘[31, 32]。

然而，LLM 很难一次性持续生成正确的行动代码 [41]。为了应对这一挑战，我们提出了一种**迭代提示机制**，它可以(1) 执行生成的程序，从 Minecraft 模拟中获取观察结果（如库存清单和附近的生物），并从代码解释器中获取错误跟踪（如有）；(2) 将反馈纳入 GPT-4 的提示中，进行新一轮的代码完善；(3) 重复该过程，直到自我验证模块确认任务完成，此时我们将程序提交到技能库（如 `craftStoneShovel()` 和 `combatZombieWithSword()`），并查询下一个里程碑的自动课程（图 2）。

经验表明，VOYAGER 具有很强的**在情境中终身学习**的能力。它可以构建一个不断增长的行动程序技能库，这些程序可重复使用、可解释，并可推广到新任务中。我们在开源 Minecraft 人工智能框架 MineDojo [23] 中对 VOYAGER 与其他基于 LLM 的代理技术（如 ReAct [29]、Reflexion [30]、AutoGPT [28]）进行了系统评估。VOYAGER 的表现优于之前的 SOTA，它获得的独特物品比 SOTA 多 3.3×，解锁关键技术里程碑的速度比 SOTA 快 15.3×，穿越的距离比 SOTA 长 2.3×。我们进一步证明，VOYAGER 能够在新的 Minecraft 世界中利用学习到的技能库从头开始解决新任务，而其他方法则很难做到这一点。

2 方法

VOYAGER 由三个新颖的部分组成：(1)自动课程（第 2.1 节），为开放式探索提出目标；(2)技能库（第 2.2 节），用于开发日益复杂的行为；(3)迭代提示机制（第 2.3 节），生成可执行代码，用于嵌入式控制。完整的提示信息见附录 A 部分。

2.1 自动课程

在开放式环境中，实体代理会遇到各种复杂程度不同的目标。自动课程为开放式探索提供了许多好处，确保学习过程具有挑战性但易于管理，培养代理学习和探索的好奇心驱动内在动力，并鼓励开发通用和灵活的问题解决策略[42-44]。我们的自动课程利用了 GPT-4 中包含的互联网规模的知识，通过提示它提供源源不断的新任务或挑战。课程以自下而上的方式展开，对探索进度和代理的当前状态具有相当大的适应性和响应性（图 3）。随着 VOYAGER 逐步实现更难的自我驱动目标，它自然会学习各种技能，例如“开采钻石”。

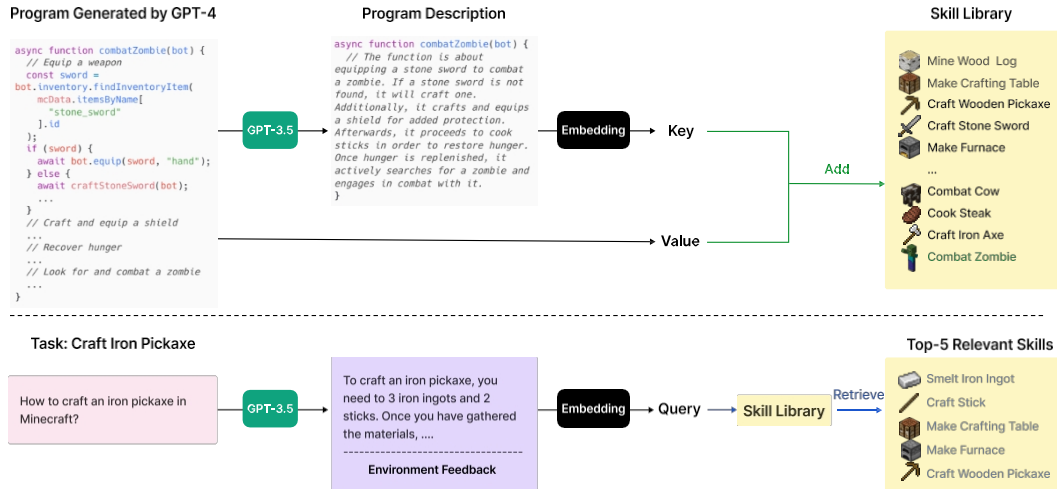


图 4：技能库。上图：添加新技能。每次 GPT-4 生成并验证一项新技能时，我们都会将其添加到技能库中，技能库由一个向量数据库表示。键是程序描述的嵌入向量（由 GPT-3.5 生成），而值则是程序本身。底部：技能检索。面对自动课程提出的新任务，我们首先利用 GPT-3.5 生成解决任务的一般建议，并将其与环境反馈相结合，作为查询上下文。随后，我们进行查询，找出前 5 项相关技能。

GPT-4 的输入提示由几个部分组成：

- (1) **鼓励多样化行为并施加限制的指令**，如 "我的最终目标是发现尽可能多的多样化事物 ... 下一个任务不应该太难，因为我可能还没有必要的资源或学到足够的技能来完成它"；
- (2) **代理的当前状态**，包括库存、装备、附近的区块和实体、生物群落、时间、健康条和饥饿条以及位置；
- (3) **之前完成和失败的任务**，反映了代理当前的探索进度和能力边界；
- (4) **其他背景**：我们还利用 GPT-3.5 根据代理的当前状态和探索进度自我提问，并自我回答问题。出于预算考虑，我们选择使用 GPT-3.5 而不是 GPT-4 来完成标准 NLP 任务。

2.2 技能库

随着自动课程不断提出日益复杂的任务，必须建立一个技能库，作为学习和进化的基础。受程序的通用性、互拟性和普遍性的启发[45]，我们用可执行代码来表示每种技能，这些代码可以在完成自动课程提出的特定任务时，对时间上的扩展动作进行扫描。

GPT-4 的输入提示由以下部分组成：

- (1) **代码生成指南**，如 "您的函数将在构建更复杂的函数时被重复使用。因此，您应使其具有通用性和可重用性"；
- (2) **控制原始应用程序接口，以及从技能库中获取的相关技能**，这些对于情境学习[36-38]的顺利进行至关重要；
- (3) **上一轮生成的代码、环境反馈、执行错误和评论**，GPT-4 可在此基础上进行自我改进（第 2.3）；
- (4) **代理的当前状态**，包括库存、装备、附近的区块和实体、生物群落、时间、健康条和饥饿条以及位置；

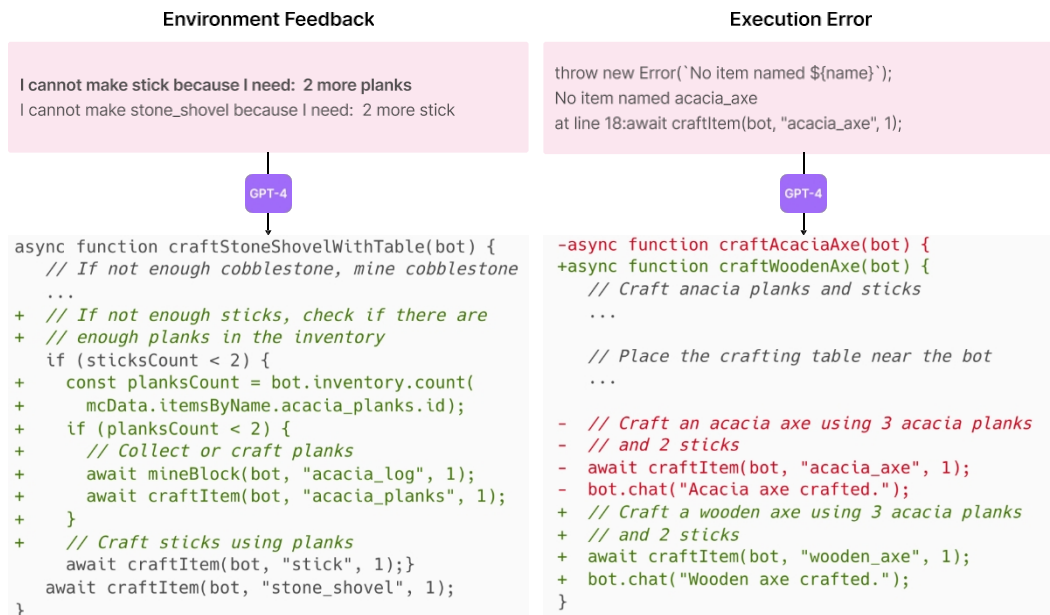


图 5: 左: 环境反馈。GPT-4 意识到在制作木棍之前还需要 2 块木板。右图执行错误。GPT-4 意识到应该制作木斧而不是金合欢斧, 因为 Minecraft 中没有金合欢斧。为简洁起见, 我们只显示了部分提示。代码生成的完整提示结构见附录 A.4 节。

(5) 思维链提示 [46] 在代码生成之前进行推理。

我们通过新颖的迭代提示机制 (第 2.3) 对节程序进行迭代改进, 将其作为一项新技能纳入技能库, 并通过其描述的嵌入对其进行索引 (图 4, 上部)。在技能检索方面, 我们通过嵌入自我生成的任务计划和环境反馈来查询技能库 (图 4, 下部)。通过不断扩大和完善技能库, VOYAGER 可以学习、适应和胜任各种任务, 在开放世界中不断挑战自身能力的极限。

2.3 迭代提示机制

我们通过三种类型的反馈引入了自我改进的迭代提示机制:

- (1) **环境反馈**, 说明程序执行的中间进度 (图 5, 左)。例如, "我无法制作铁制胸甲, 因为我需要.....":
再加 7 块铁锭 "突出了制作铁胸甲失败的原因。我们在控制原始应用程序接口内使用 bot.chat() 生成环境反馈, 并提示 GPT-4 在代码生成过程中也使用该功能;
- (2) 来自程序解释器的**执行错误**, 揭示程序中的任何无效操作或语法错误, 这对修复错误很有价值 (图 5, 右);
- (3) **检查任务成功与否的自我验证**。我们不需要为自动课程提出的每个新任务手动编码成功检查器, 而是将另一个 GPT-4 代理实例化, 用于自我验证。通过向 GPT-4 提供 VOYAGER 的当前状态和任务, 我们要求它扮演批评者的角色 [47-49], 告知我们程序是否完成了任务。此外, 如果任务失败, 它还会提出批评, 建议如何完成任务 (图 6)。因此, 我们的自我验证比自我反思[30]更全面, 既能检查成功, 又能反思错误。

在每一轮代码生成过程中, 我们都会执行生成的程序, 从代码解释器中获取环境反馈和执行错误, 并将其纳入 GPT-4 的提示中, 以便进行下一轮代码完善。这个迭代过程会不断重复, 直到自我验证验证通过为止。

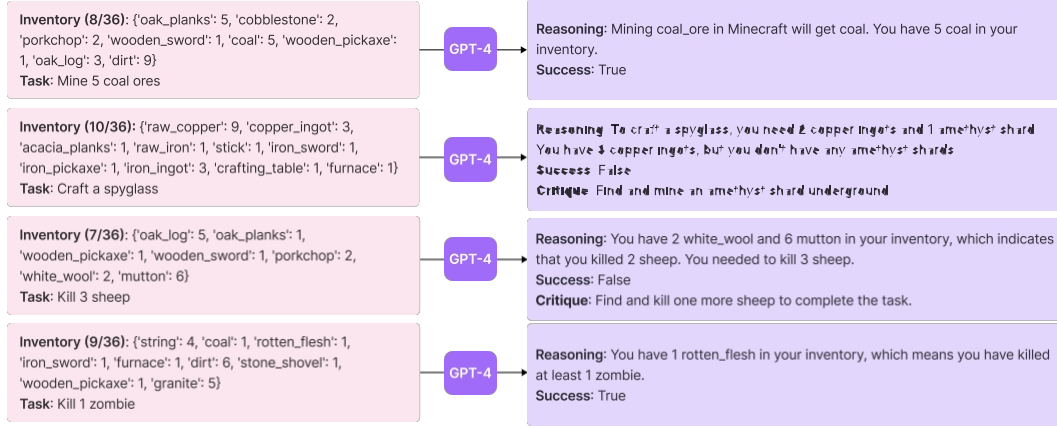


图 6：自我验证示例。为简洁起见，我们只显示了部分提示。完整的提示结构请参见附录第 A.5 节。

任务完成后，我们会将新技能添加到技能库中，并要求自动课程表提供新目标（图 2）。如果代理在 4 轮代码生成后卡住了，我们就会向课程表询问另一个任务。这种迭代提示方法大大改进了嵌入式控制的程序合成，使 VOYAGER 能够在无需人工干预的情况下不断获得各种技能。

3 实验

3.1 实验设置

我们利用 OpenAI 的 gpt-4-0314 [35] 和 gpt-3.5-turbo-0301 [50] API 进行文本补全，并利用 text-embedding-ada-002 [51] API 进行文本嵌入。我们将所有温度设置为 0，但自动课程除外，该课程使用温度=0.1 以鼓励任务多样性。我们的模拟环境建立在 MineDojo [23] 的基础之上，并利用 Mineflayer [52] JavaScript API 进行电机控制。详见附录 B.1 节。

3.2 基准线

由于目前还没有基于 LLM 的代理可以直接用于 Minecraft，因此我们尽最大努力选择了一些具有代表性的算法作为基准。这些方法最初只针对无体现的 NLP 任务而设计，因此我们必须对其进行重新解释，使其能够在 MineDojo 中执行，并与我们的实验设置兼容：

ReAct [29] 使用思维链提示 [46]，通过 LLM 生成推理轨迹和行动计划。我们将环境反馈和代理状态作为观察结果提供给它。

Reflexion [30] 建立在 ReAct [29] 的基础上，通过自我反思来推断更直观的未来行动。我们为其提供了执行错误和自我验证模块。

AutoGPT [28] 是一款流行的软件工具，它通过将高级目标分解为多个子目标，并在 ReAct 式的循环中执行这些子目标，从而实现 NLP 任务的自动化。我们使用 GPT-4 对 AutoGPT 进行任务分解，并提供代理状态、环境反馈和执行错误作为子目标执行的观察结果。与 VOYAGER 相比，AutoGPT 缺少用于积累知识的技能库、用于评估任务成功与否的自我验证以及用于开放式探索的自动课程。

需要注意的是，我们并没有直接与之前将 Minecraft 屏幕像素作为输入并输出低级控制的方法进行比较 [53-55]。这不是苹果与苹果之间的比较，因为我们依赖高级 Mineflayer [52] 应用程序接口来控制代理。我们的工作重点是突破 GPT-4 的极限，实现终生嵌入式代理学习，而不是解决三维感知或传感运动控制问题。VOYAGER 是正交的，可以与基于梯度的方法相结合，如

表 1: 技术树掌握情况。分数表示总运行三次中成功的次数。0/3 表示该方法未能在最大提示迭代次数（160 次）内解锁技术树的一个等级。

(160). 数字是三次试验的平均提示迭代次数。迭代次数越少，该方法的效率越高。

方法	木制工具	石制工具	铁制工具	金刚石工具
再行动 [29]	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)
反射 [30]	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)
AutoGPT [28]	92± 72 (3/3)	94± 72 (3/3)	135± 103 (3/3)	不适用 (0/3)
无技能库的 VOYAGER	7± 2 (3/3)	9± 4 (3/3)	29± 11 (3/3)	不适用 (0/3)
VOYAGER (Ours)	6± 2 (3/3)	11± 2 (3/3)	21± 7 (3/3)	102 (1/3)

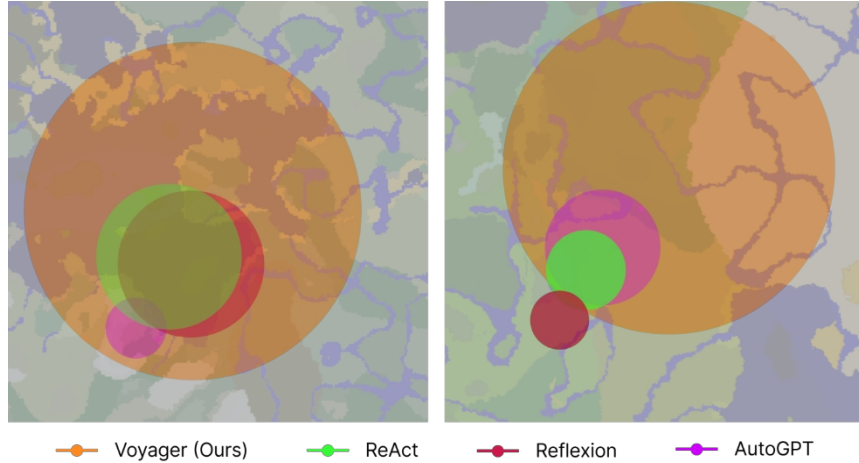


图 7: 地图覆盖范围: Minecraft 地图鸟瞰图。VOYAGER 能够穿越 2.3× 与基线相比, VOYAGER 能够穿越更远的距离, 同时穿越各种地形。

只要控制器提供代码应用程序接口, VPT [8]就能实现。我们在表 A.2 中对 VOYAGER 和之前的 Minecraft 代理进行了系统级比较。A.2.

3.3 评估结果

我们系统地评估了 VOYAGER 和基线的探索性能、科技树掌握程度、地图覆盖范围以及在新世界中执行新任务的零点泛化能力。

勘探效果显著提高。探索性能结果如图 1。所示 VOYAGER 的优越性体现在它能够不断取得新进展, 在 160 次提示迭代中发现了 63 个独特的项目, 比同类产品多出 3.3× 个新项目。另一方面, AutoGPT 在发现新项目方面落后很多, 而 ReAct 和 Reflexion 则很难取得显著进展, 因为开放式探索目标的抽象性很强, 如果没有合适的课程, 执行起来很有挑战性。

持续掌握技术树。Minecraft 技术树测试代理制作和使用工具的能力。要在这棵树（木制工具→ 石制工具→ 铁制工具→ 钻石工具）中取得进展, 需要代理掌握系统和组合技能。与基线相比, VOYAGER 解锁木制工具等级的速度快 15.3×（以提示迭代次数计算）, 石制工具等级的速度快 8.5×, 铁制工具等级的速度快 6.4×, 而且 VOYAGER 是唯一一个解锁技术树钻石等级的工具（图 2 和表 1）。这凸显了自动课程的有效性, 它不断提出具有适当复杂性的挑战, 以促进代理的进步。

广泛的地图穿越。通过穿越各种地形, VOYAGER 的导航距离比基线多出 2.3×, 而基线代理经常发现自己被局限在局部区域, 这大大削弱了它们发现新知识的能力（图 7）。

表 2：对未见任务的零点泛化。分数表示三次尝试中成功的次数。0/3 表示该方法未能在最大提示迭代次数（50）内解决任务。数字是三次试验的平均提示迭代次数。迭代次数越少，说明该方法越有效。

方法	金刚镐	金剑	熔岩桶	指南针
再行动 [29]	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)
反射 [30]	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)
AutoGPT [28]	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)	不适用 (0/3)
AutoGPT [28] w/ 我们的技能库	39 (1/3)	30 (1/3)	不适用 (0/3)	30 (2/3)
不含技能库的 VOYAGER	36 (2/3)	30±9 (3/3)	27±9 (3/3)	26±3 (3/3)
VOYAGER (我们的)	19±3 (3/3)	18±7 (3/3)	21±5 (3/3)	18±2 (3/3)

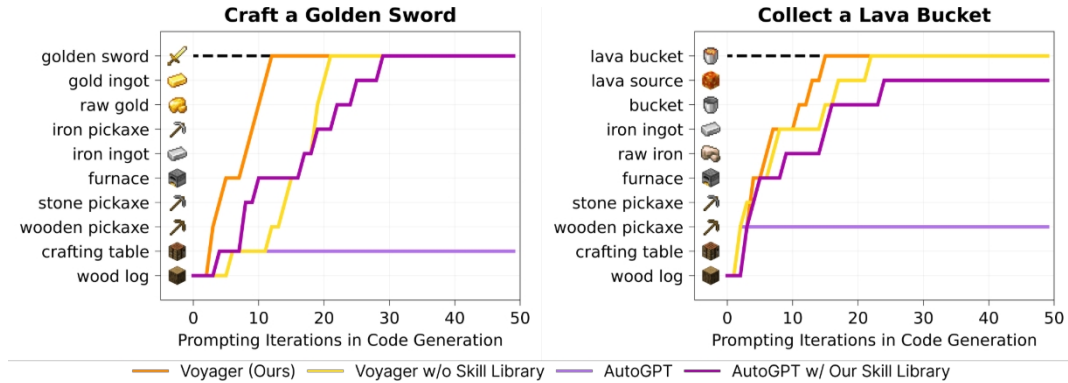


图 8：对未见任务的零点泛化。我们展示了每种方法在两个任务上的中间进展。其他两项任务见附录 B.4.3 节。我们没有绘制 ReAct 和 Reflexion，因为它们没有取得任何有意义的进展。

对未知任务进行高效的零点泛化。为了评估零点泛化，我们清除了代理的库存，将其重置到一个新的实例化世界，并用未见任务对其进行测试。对于 VOYAGER 和 AutoGPT，我们利用 GPT-4 将任务分解为一系列子目标。表 2 和图 8 显示，VOYAGER 可以稳定地解决所有任务，而基线则无法在 50 次提示迭代内解决任何任务。值得注意的是，我们从终身学习中构建的技能库不仅提高了 VOYAGER 的性能，也为 AutoGPT 带来了动力。这表明，技能库是一种多功能工具，可随时用于其他方法，是一种即插即用的资产，可有效提高性能。

3.4 消融研究

我们在 VOYAGER 中消融了 6 种设计选择（自动课程、技能库、环境反馈、执行错误、自我验证和用于代码生成的 GPT-4），并研究了它们对探索性能的影响（每种消融变体的详细信息请参见附录 B.3 节）。结果如图 9 所示。我们将在下文重点介绍主要发现：

- **自动课程设置对代理的持续进步至关重要。**如果用随机课程表代替自动课程表，发现的物品数量会减少 93%，因为如果不按顺序尝试某些任务，可能会过于艰巨。另一方面，手动设计的课程需要大量 Minecraft 特定的专业知识，而且没有考虑到代理的实际情况。与我们的自动课程相比，手动课程的实验结果并不理想。
- **没有技能库的《VOYAGER》在后期阶段表现出趋于平稳的趋势。**这凸显了技能库在 VOYAGER 中的关键作用。它有助于创建更复杂的动作，并通过鼓励在旧技能的基础上开发新技能，稳步推进代理的极限。

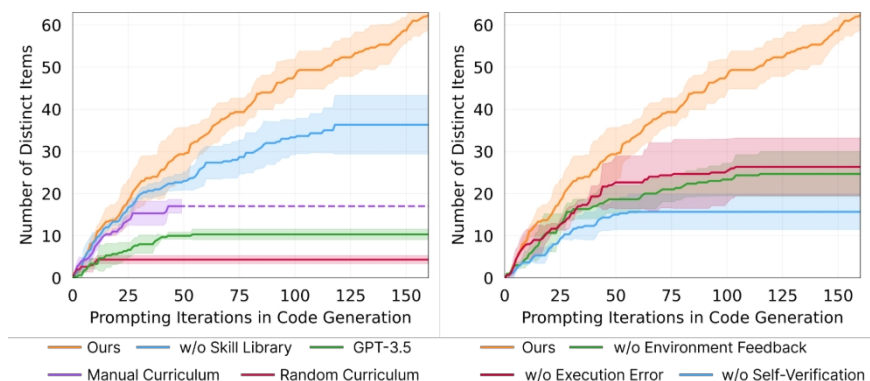


图 9：左：自动课程、技能库和 GPT-4 的消融研究。GPT-3.5 表示用 GPT-3.5 代替 GPT-4 生成代码。VOYAGER 的表现优于所有替代方案，证明了每个组件的关键作用。右图：迭代提示机制的消融研究。VOYAGER 超越了所有其他方案，从而突出了迭代提示机制中每种反馈的重要意义。



图 10：VOYAGER 利用人类反馈构建三维结构。从左到右展示了结合人类输入的建筑设计的进展。

- 在所有反馈类型中，自我验证最为重要。移除该模块会导致被发现的项目数量大幅下降（-73%）。自我验证是决定何时继续执行新任务或重新尝试之前未成功任务的关键机制。
- 在代码生成方面，GPT-4 明显优于 GPT-3.5，获得的唯一一项比 GPT-3.5 多 5.7 $\times$ ，因为 GPT-4 在编码能力上有了质的飞跃。这一发现与最近的文献研究[56, 57]相吻合。

3.5 来自人类的多模态反馈

VOYAGER 目前不支持视觉感知，因为在撰写本文时，现有版本的 GPT-4 API 仅支持文本。不过，VOYAGER 有潜力通过多模态感知模型[58, 59]来实现更令人印象深刻的任务。我们演示了在人类反馈的情况下，VOYAGER 能够在 Minecraft 中构建复杂的三维结构，例如尼瑟斯传送门和房屋（图 10）。整合人类反馈有两种方法：

- (1) 人类作为评论员（相当于 VOYAGER 的自我验证模块）：人类向 VOYAGER 提供视觉评论，使其能够修改上一轮的代码。这种反馈对于纠正 VOYAGER 无法直接感知的三维结构空间细节中的某些错误至关重要。
- (2) 人类作为课程（相当于 VOYAGER 的自动课程模块）：人类将复杂的建筑任务分解成更小的步骤，引导 VOYAGER 逐步完成这些步骤。这种方法提高了 VOYAGER 处理更复杂的 3D 建筑任务的能力。

4 局限性和未来工作

成本。GPT-4 应用程序接口的成本很高。它比 GPT-3.5 昂贵 15 $\times$ 。然而，VOYAGER 需要 GPT-4 在代码生成质量上的飞跃（图 9），而 GPT-3.5 和开源 LLM 无法提供这种飞跃 [60]。

不准确性。尽管有迭代提示机制，但仍会出现代理被卡住、无法生成正确技能的情况。自动课程可以灵活地在以后重新尝试这项任务。自我验证模块有时也会失效，例如无法识别蜘蛛串是击败蜘蛛的成功信号。

幻觉。自动课程偶尔会提出一些无法完成的任务。例如，它可能会要求代理制作“铜剑”或“铜胸甲”，而这些物品在游戏中是不存在的。代码生成过程中也会出现幻觉。例如，GPT-4 倾向于使用鹅卵石作为燃料输入，尽管这在游戏中是无效的燃料来源。此外，它可能会调用所提供的控制原始 API 中不存在的函数，从而导致代码执行错误。

我们相信，GPT API 模型的改进以及对开源 LLM 进行微调的新技术将在未来克服这些限制。

5 相关工作

Minecraft 中的决策制定代理。Minecraft 是一个开放式三维世界，游戏机制极其灵活，支持多种活动。基于著名的 Minecraft 基准 [23, 61-65]，Minecraft 学习算法可分为两类：1) 低级控制器：许多先前的研究利用分层强化学习来学习人类演示 [66-68]。Kanitscheider 等人[14] 根据成功率设计课程，但其目标仅限于策划好的项目。MineDojo [23] 和 VPT [8] 利用 YouTube 视频进行大规模预训练。DreamerV3 [69] 则通过学习世界模型来探索环境和收集钻石。2) 高级计划器：Volum 等人[70] 利用 Codex [41] 的少量提示来生成可执行的策略，但他们需要额外的人机交互。最近的研究利用 LLMs 作为 Minecraft 中的高级计划器，按照 Minecraft 配方将高级任务分解为多个子目标[55, 53, 71]，因此缺乏充分的探索灵活性。与后几部作品一样，VOYAGER 也通过提示 GPT-4 将 LLMs 用作高级规划器，并按照 Volum 等人[70]的做法利用 Mineflayer [52] 作为低级控制器。与之前的作品不同，VOYAGER 采用的是一种自动课程，以自下而上的方式展开，由好奇心驱动，因此可以进行开放式探索。

用于代理规划的大型语言模型。受 LLMs 强大的新兴能力（如零触发提示和复杂推理）[72, 37, 38, 36, 73, 74]的启发，具身代理研究[75-78]见证了将 LLMs 用于规划目的的显著增加。最近的研究大致可分为两类。1) 用于机器人学习的大型语言模型：之前的许多研究都将 LLMs 用于生成机器人规划的子目标 [27, 27, 25, 79, 80]。Inner Monologue [26] 将环境反馈与 LLMs 结合起来，用于机器人规划。Code as Policies [16] 和 ProgPrompt [22] 直接利用 LLM 生成可执行的机器人策略。VIMA [19] 和 PaLM-E [59] 对预先训练的 LLM 进行了微调，以支持多模态提示。2) 用于文本代理的大型语言模型：ReAct [29] 利用思维链提示[46]，用 LLM 生成推理轨迹和特定任务行动。Reflexion [30] 建立在 ReAct [29] 的基础上，通过自我反思来增强推理能力。AutoGPT[28]是一种流行的工具，它通过为一个高层次目标而设计由多个子目标组成的课程，同时结合 ReAct[29]的推理和行动循环，从而实现 NLP 任务的自动化。DERA [81] 将任务构建为两个 GPT-4 [35] 代理之间的对话。生成式代理[82]利用 ChatGPT[50]模拟人类行为，将代理的经验存储为记忆，并检索这些记忆进行规划，但其代理行动不可执行。SPRING [83] 是一项并行工作，它使用 GPT-4 从游戏手册中提取游戏机制，并在此基础上回答排列在有向无环图中的问题，预测下一步行动。所有这些作品都缺乏开发更复杂行为的技能库，而这正是 VOYAGER 在终身学习方面取得成功的关键要素。

代码生成与执行。代码生成一直是 NLP 领域的一项长期挑战 [41, 84, 85, 73, 37]。

综合。执行指导法利用中间执行结果来指导程序搜索 [86-88]。另一个研究方向是利用多数投票来根据执行性能选择候选程序 [89, 90]。此外，LEVER [91] 训练验证器根据执行结果区分和拒绝不正确的程序。另一方面，CLAIRIFY [92] 为规划化学实验生成代码，并利用基于规则的验证器向 LLMs 迭代提供错误反馈。VOYAGER 将环境反馈、执行错误和自我验证（用于评估任务成功与否）整合到一个迭代提示机制中，从而实现了嵌入式控制。

6 结论

在这项工作中，我们介绍了第一个由 LLM 驱动的嵌入式终身学习代理 VOYAGER，它利用 GPT-4 持续探索世界，开发出越来越复杂的技能，并在没有人类干预的情况下不断有新的发现。VOYAGER 在发现新奇物品、解锁 Minecraft 技术树、穿越各种地形以及在新实例化的世界中将学到的技能库应用于未知任务方面表现出卓越的性能。VOYAGER 是开发功能强大的通用代理的起点，无需调整模型参数。

7 更广泛的影响

我们的研究是在 Minecraft 这个安全无害的 3D 视频游戏环境中进行的。虽然 VOYAGER 的设计普遍适用于机器人等其他领域，但将其应用于物理机器人需要更多关注，并由人类实施安全约束，以确保责任和安全的部署。

8 鸣谢

我们非常感谢朱子明、杨开宇、Rafał Kocielnik、Colin White、Or Sharir、Sahin Lale、黄德安、Jean Kossaifi、杨云聪、张锟、黄敏超以及其他许多同事和朋友的有益反馈和深入讨论。这项工作是王冠之在英伟达公司实习期间完成的。王冠之得到了加州理工学院计算与数学科学系 Kortschak 奖学金的资助。

参考文献

- [1] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Daniel Gordon, Yuke Zhu, Abhinav Gupta, and Ali Farhadi. Ai2-thor: Ai2-thor: An interactive 3d environment for visual ai. *ArXiv preprint arXiv: 1712.05474*, 2017.
- [2] Manolis Savva, Jitendra Malik, Devi Parikh, Dhruv Batra, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu 和 Vladlen Koltun. 栖息地：具身人工智能研究平台。In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*, pages 9338-9346. IEEE, 2019.
- [3] Yuke Zhu, Josiah Wong, Ajay Mandlekar 和 Roberto Martín-Martín: *ArXiv preprint arXiv: 2009.12293*, 2020.
- [4] Fei Xia, William B. Shen, Chengshu Li, Priya Kasimbeg, Micael Tchammi, Alexander Toshev, Li Fei-Fei, Roberto Martín-Martín 和 Silvio Savarese. 交互式吉布森基准 (igibson 0.5): *ArXiv preprint arXiv: 1910.14442*, 2019.
- [5] Bokui Shen, Fei Xia, Chengshu Li, Roberto Martín-Martín, Linxi Fan, Guanzhi Wang, Claudia Pérez-D'Arpino, Shyamal Buch, Sanjana Srivastava, Lyne P. Tchammi, Micael E. Tchammi, Kent Vainio, Josiah Wong, Li Fei-Fei and Silvio Savarese. Tchammi, Kent Vainio, Josiah Wong, Li Fei-Fi Fe, and Silvio Savarese.

- [6] Jens Kober、J Andrew Bagnell 和 Jan Peters。机器人中的强化学习：A survey。*国际机器人研究期刊*，32（11）：1238-1274，2013 年。
- [7] Kai Arulkumaran、Marc Peter Deisenroth、Miles Brundage 和 Anil Anthony Bharath。深度强化学习：简要调查。*IEEE 信号处理杂志*，34（6）：26-38，2017。
- [8] Bowen Baker、Ilge Akkaya、Peter Zhokhov、Joost Huizinga、Jie Tang、Adrien Ecoffet、Brandon Houghton、Raul Sampedro 和 Jeff Clune。视频预训练（vpt）：*ArXiv preprint arXiv: Arxiv-2206.11795*，2022。
- [9] DeepMind 交互式代理团队、Josh Abramson、Arun Ahuja、Arthur Brusee、Federico Carnevale、Mary Cassin、Felix Fischer、Petko Georgiev、Alex Goldin、Mansi Gupta、Tim Harley、Felix Hill、Peter C Humphreys、Alden Hung、Jessica Landon、Timothy Lillicrap、Hamza Merzic、Alistair Muldal、Adam Santoro、Guy Scully、Tamara von Glehn、Greg Wayne、Nathaniel Wong、Chen Yan 和 Rui Zhu。利用模仿和自我监督学习创建多模态交互式代理》，*arXiv preprint arXiv: Arxiv-2112.03763*，2021。
- [10] Oriol Vinyals, Igor Babuschkin, Junyoung Chung, Michael Mathieu, Max Jaderberg, Wojciech M Czarnecki, Andrew Dudzik, Aja Huang, Petko Georgiev, Richard Powell, et al. Alphastar: 掌握即时战略游戏《星际争霸II》。*DeepMind 博客*，2019 年 2 月。
- [11] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *ArXiv preprint arXiv: Arxiv-1901.10995*, 2019.
- [12] Joost Huizinga 和 Jeff Clune. 用组合多目标进化算法通过许多垫脚石进化多模态机器人行为。*进化计算*，30（2）：131-164，2022。
- [13] 王锐、乔尔-雷曼、阿迪提亚-拉瓦尔、支嘉乐、李玉仑、杰弗里-克鲁尼和肯尼斯-O-斯坦利。增强型 POET：通过无限制地发明学习挑战及其解决方案进行开放式强化学习。In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 9940-9951. PMLR, 2020.
- [14] Ingmar Kanitscheider, Joost Huizinga, David Farhi, William Hebgen Guss, Brandon Houghton, Raul Sampedro, Peter Zhokhov, Bowen Baker, Adrien Ecoffet, Jie Tang, Oleg Klimov, and Jeff Clune. 复杂、可视、难探索领域中的多任务课程学习：*ArXiv preprint arXiv: Arxiv-2106.14876*，2021。
- [15] Michael Dennis、Natasha Jaques、Eugene Vinitzky、Alexandre M. Bayen、Stuart Russell、Andrew Critch 和 Sergey Levine。通过无监督环境设计实现新兴复杂性和零点转移。在雨果-拉罗谢尔、马克-奥雷利奥-兰扎托、拉亚-哈德塞尔、玛丽亚-弗洛里娜-巴尔坎和宣天林主编的《*神经信息处理系统进展 33：2020 年神经信息处理系统年会*》（*NeurIPS 2020*）中，2020 年 12 月 6-12 日，虚拟，2020 年。
- [16] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 代码即政策：*ArXiv preprint arXiv: Arxiv-2209.07753*，2022。
- [17] Shao-Hua Sun, Te-Lin Wu, and Joseph J. Lim. 程序引导的代理。*第八届学习表征国际会议, ICLR 2020, 埃塞俄比亚的斯亚贝巴, 2020 年 4 月 26-30 日*。OpenReview.net, 2020.
- [18] Zelin Zhao, Karan Samel, Binghong Chen, and Le Song. Proto：程序引导任务的程序引导转换器。In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 17021-17036, 2021.
- [19] 蒋云帆、阿格里姆-古普塔、张子辰、王冠之、龚永强、陈彦君、李菲菲、阿尼玛-阿南德库马尔、朱玉珂、范林熙（Jim）。维玛：多模态提示下的通用机器人操纵。*arxiv.org*，2022 年。

- [20] Mohit Shridhar, Lucas Manuelli 和 Dieter Fox. Cliport: *ArXiv preprint arXiv: Arxiv-2109.12098*, 2021.
- [21] Linxi Fan, Guanzhi Wang, De-An Huang, Zhiding Yu, Li Fei-Fi Fei, Yuke Zhu, and Animashree Anandkumar. SECANT: 视觉策略零镜头泛化的自我专家克隆。见 Marina Meila 和 Tong Zhang 编辑的《第 38 届机器学习国际会议论文集》(ICML 2021, 2021 年 7 月 18-24 日, 虚拟活动), 《机器学习研究论文集》第 139 卷, 第 3088-3099 页。PMLR, 2021.
- [22] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: *ArXiv preprint arXiv: Arxiv-2209.11302*, 2022.
- [23] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *ArXiv preprint arXiv: Arxiv-2206.08853*, 2022.
- [24] Andy Zeng, Adrian Wong, Stefan Welker, Krzysztof Choromanski, Federico Tombari, Aveek Purohit, Michael Ryoo, Vikas Sindhwani, Johnny Lee, Vincent Vanhoucke 和 Pete Florence. 苏格拉底模型: *arXiv preprint arXiv: Arxiv-2204.00598*, 2022.
- [25] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, 德米特里-卡拉什尼科夫、匡宇恒、李光辉、谢尔盖-莱文、卢瑶、卢琳达、卡罗琳娜-帕拉达、彼得-帕斯托尔、约内尔-基安博、卡尼什卡-拉奥、贾雷克-雷廷豪斯、迭戈-雷耶斯、皮埃尔-塞曼内、尼古拉斯-西弗斯、克莱顿-谭、亚历山大-托舍夫、文森特-范豪克、夏飞、肖特、徐鹏、徐思春和颜梦媛。做我能做的, 而不是我说的: *ArXiv preprint arXiv: Arxiv-2204.01691*, 2022.
- [26] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. 内心独白: *ArXiv preprint arXiv: Arxiv- 2207.05608*, 2022.
- [27] 黄文龙、Pieter Abbeel、Deepak Pathak 和 Igor Mordatch. 语言模型作为 "零射击规划者": 为具身代理提取可操作知识。In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pages 9118-9147. PMLR, 2022.
- [28] Significant-gravitas/auto-gpt: 使 gpt-4 完全自动化的实验性开源尝试, 2023 年。
- [29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 反应: *arXiv preprint arXiv: Arxiv-2210.03629*, 2022.
- [30] Noah Shinn, Beck Labash 和 Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *ArXiv preprint arXiv: Arxiv-2303.11366*, 2023.
- [31] German Ignacio Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. 神经网络的持续终身学习: 回顾. *神经网络*, 113:54-71, 2019.
- [32] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. 持续学习的全面考察: *ArXiv preprint arXiv: Arxiv-2302.00487*, 2023.
- [33] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra 和 Martin Riedmiller. 用深度强化学习玩 Atari. *ArXiv 预印本: Arxiv-1312.5602*, 2013 年。

- [34] OpenAI, :, Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław De biak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique P. O. Pinto, D. O. Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski 和 Susan Zhang. 使用大规模深度强化学习的 Dota 2。 *ArXiv 预印本: Arxiv-1912.06680*, 2019 年。
- [35] OpenAI. *ArXiv preprint arXiv: Arxiv-2303.08774*, 2023.
- [36] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 大型语言模型的新兴能力。 *ArXiv 预印本 arXiv: Arxiv-2206.07682*, 2022.
- [37] Tom B. Brown、 Benjamin Mann、 Nick Ryder、 Melanie Subbiah、 Jared Kaplan、 Prafulla Dhariwal、 Arvind Neelakantan、 Pranav Shyam、 Girish Sastry、 Amanda Askell、 Sandhini Agarwal、 Ariel Herbert-Voss、 Gretchen Krueger、 Tom Henighan、 Rewon Child、 Aditya Ramesh、 Daniel M. Ziegler、 Jeff Wu、 Clemens Winter、 Christopher Hesse、 Mark Chen、 Eric Sigler、 Mateusz Litwin、 Scott Gray。 Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 语言模型是少量学习者。在雨果-拉罗谢尔、马克-奥雷利奥-兰扎托、拉伊娅-哈德塞耳、玛丽亚-弗洛里娜-巴尔坎和宣天林主编的《*神经信息处理系统进展33: 2020年神经信息处理系统年会*》 (*NeurIPS 2020*) 中, 2020年12月6-12日, 虚拟, 2020年。
- [38] Colin Raffel、 Noam Shazeer、 Adam Roberts、 Katherine Lee、 Sharan Narang、 Michael Matena、 Yanqi Zhou、 Wei Li 和 Peter J. Liu. 用统一的文本到文本转换器探索迁移学习的极限。 *J. Mach.Learn.*21:140:1-140:67, 2020.
- [39] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. 多样性就是你所需要的一切: 没有奖励函数的技能学习。 In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.
- [40] Edoardo Conti、 Vashisht Madhavan、 Felipe Petroski Such、 Joel Lehman、 Kenneth O. Stanley 和 Jeff Clune. 通过寻找新奇的代理群体改进深度强化学习进化策略中的探索。 In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 5032-5043, 2018.
- [41] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser、 穆罕默德-巴伐利亚、 克莱门斯-温特、 菲利普-蒂莱、 费利佩-佩特罗斯基-苏奇、 戴夫-卡明斯、 马蒂亚斯-普拉珀特、 福蒂奥斯-昌特齐斯、 伊丽莎白-巴恩斯、 阿里尔-赫伯特-沃斯、 威廉-赫伯根-古斯、 亚历克斯-尼科尔、 亚历克斯-帕诺、 尼古拉斯-特扎克、 唐杰、 伊戈尔-巴布什金、 苏奇尔-巴拉吉、 尚塔努-詹恩、 威廉-桑德斯、 克里斯托弗-赫塞、 安德鲁-N.Carr、 Jan Leike、 Josh Achiam、 Vedant Misra、 Evan Morikawa、 Alec Radford、 Matthew Knight、 Miles Brundage、 Mira Murati、 Katie Mayer、 Peter Welinder、 Bob McGrew、 Dario Amodei、 Sam McCandlish、 Ilya Sutskever 和 Wojciech Zaremba. 评估基于代码训练的大型语言模型。 *ArXiv 预印本 arXiv: Arxiv- 2107.03374*, 2021.
- [42] Rui Wang、 Joel Lehman、 Jeff Clune 和 Kenneth O. Stanley. 成对开放式开拓者 (诗人): 无休止地生成日益复杂多样的学习环境及其解决方案》。 *ArXiv 预印本: Arxiv-1901.01753*, 2019 年。
- [43] Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. 深度 RL 的自动课程学习: 简短调查。载于 Christian Bessiere 编辑的《*第二十九届国际人工智能联合会议论文集*》 (*IJCAI 2020*) , 第 4819-4825 页。 ijcai.org, 2020 年。

- [44] Sébastien Forestier, Rémy Portelas, Yoan Mollard, and Pierre-Yves Oudeyer. 自动课程学习的内在动机目标探索过程。《机器学习研究期刊》，23（1）：6818-6858，2022 年。
- [45] Kevin Ellis、Catherine Wong、Maxwell Nye、Mathias Sable-Meyer、Luc Cary、Lucas Morales、Luke Hewitt、Armando Solar-Lezama，以及 Joshua B. Tenenbaum。Dreamcoder：用唤醒-睡眠贝叶斯程序学习增长可概括、可解释的知识。 *ArXiv 预印本 arXiv: Arxiv-2006.08381*, 2020.
- [46] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. 大型语言模型中的思维链提示推理。 *ArXiv preprint arXiv: Arxiv-2201.11903*, 2022.
- [47] Volodymyr Mnih、Adrià Puigdomènech Badia、Mehdi Mirza、Alex Graves、Timothy P. Lillicrap、Tim Harley、David Silver 和 Koray Kavukcuoglu。深度强化学习的异步方法。 In Maria-Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1928-1937. JMLR.org, 2016.
- [48] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. *ArXiv preprint arXiv: Arxiv-1707.06347*, 2017.
- [49] Timothy P. Lillicrap、Jonathan J. Hunt、Alexander Pritzel、Nicolas Heess、Tom Erez、Yuval Tassa、David Silver 和 Daan Wierstra。深度强化学习的连续控制。 In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
- [50] 介绍 chatgpt，2022。
- [51] 新改进的嵌入模型，2022 年。
- [52] PrismarineJS. Prismarinejs/mineflayer：使用强大、稳定和高级的 javascript api 创建 minecraft 机器人，2013 年。
- [53] Kolby Nottingham、Prithviraj Ammanabrolu、Alane Suhr、Yejin Choi、Hanna Hajishirzi、Sameer Singh 和 Roy Fox。具身代理会梦见像素羊吗？使用语言引导的世界建模进行化身决策。 *arxiv.org*，2023。
- [54] 蔡少飞、王子豪、马晓健、刘安吉和梁一涛。通过目标感知表征学习和自适应地平线预测的开放世界多任务控制》， *arXiv 预印本 arXiv: Arxiv-2301.10034*, 2023.
- [55] 王梓豪、蔡绍飞、刘安吉、马晓健、梁一涛。描述、解释、规划和选择：使用大型语言模型的交互式规划使开放世界多任务代理成为可能》， *arXiv preprint arXiv: Arxiv-2302.01560*, 2023.
- [56] Sébastien Bubeck、Varun Chandrasekaran、Ronen Eldan、Johannes Gehrke、Eric Horvitz、Ece Kamar、Peter Lee、Yin Tat Lee、Yuanzhi Li、Scott Lundberg、Harsha Nori、Hamid Palangi、Marco Tulio Ribeiro 和 Yi Zhang。人工通用智能的火花： *ArXiv preprint arXiv: Arxiv-2303.12712*, 2023.
- [57] 刘以恒、韩天乐、马思远、张家悦、杨媛媛、田家明、何浩、李安通、何梦申、刘正良、吴子豪、朱大江、李翔、强宁、沈定刚、刘天明、葛宝。chatgpt/gpt-4 研究综述及大型语言模型的未来展望》， *ArXiv preprint arXiv: Arxiv-2304.01852*, 2023.
- [58] Shikun Liu、Linxi Fan、Edward Johns、Zhiding Yu、Chaowei Xiao 和 Anima Anandkumar. Prism： *ArXiv preprint arXiv: Arxiv- 2303.02506*, 2023.

- [59] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: *ArXiv preprint arXiv: Arxiv-2303.03378*, 2023.
- [60] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timo-thée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave 和 Guillaume Lample. Llama: Open and efficient foundation language models. *ArXiv preprint arXiv: Arxiv-2302.13971*, 2023.
- [61] William H. Guss, Brandon Houghton, Nicholay Topin, Phillip Wang, Cayden Codel, Manuela Veloso 和 Ruslan Salakhutdinov. Minerl: 一个大规模的 minecraft 演示数据集。In Sarit Kraus, editor, *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, pages 2442-2448. ijcai.org, 2019.
- [62] William H. Guss, Cayden Codel, Katja Hofmann, Brandon Houghton, Noboru Kuno, Stephanie Milani, Sharada Mohanty, Diego Perez Liebana, Ruslan Salakhutdinov, Nicholay Topin, Manuela Veloso 和 Phillip Wang. 使用人类前验进行样本高效再强迫学习的 minerl 2019 竞赛。 *ArXiv 预印本 arXiv: Arxiv-1904.10079*, 2019.
- [63] William H. Guss, Mario Yncente Castro, Sam Devlin, Brandon Houghton, Noboru Sean Kuno, Crisman Loomis, Stephanie Milani, Sharada Mohanty, Keisuke Nakata, Ruslan Salakhutdinov, John Schulman, Shinya Shiroshita, Nicholay Topin, Avinash Ummadisingu 和 Oriol Vinyals. 使用人类先验的样本高效强化学习竞赛 (the minerl 2020 competition on sample efficient reinforcement learning using human priors) 。 *ArXiv 预印本 arXiv: Arxiv-2101.11071*, 2021.
- [64] Anssi Kanervisto, Stephanie Milani, Karolis Ramanauskas, Nicholay Topin, 林子川, Jun-you Li, Jianing Shi, Deheng Ye, Qiang Fu, Wei Yang, Weijun Hong, Zhongyue Huang, Haicheng Chen, Guangjun Zeng, Yue Lin, Vincent Micheli, Eloi Alonso, François Fleuret, Alexander Nikulin, Yury Belousov, Oleg Svidchenko 和 Aleksei Shpilman. Minerl 2021 年钻石竞赛: *ArXiv preprint arXiv: Arxiv-2202.10583*, 2022.
- [65] Matthew Johnson, Katja Hofmann, Tim Hutton 和 David Bignell. 用于人工智能实验的马尔默平台。In Subbarao Kambhampati, editor, *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, 9-15 July 2016*, pages 4246-4247. IJCAI/AAAI Press, 2016.
- [66] Zichuan Lin, Junyou Li, Jianing Shi, Deheng Ye, Qiang Fu, and Wei Yang. 觉悟-麦克: *ArXiv preprint arXiv: Arxiv-2112.04907*, 2021.
- [67] 毛瀚宇、王超、郝晓天、毛一欢、吕一鸣、吴成杰、郝建业、李东、唐平忠。世海: *ArXiv preprint arXiv: Arxiv-2111.08857*, 2021.
- [68] Alexey Skrynnik, Aleksey Staroverov, Ermek Aitygulov, Kirill Aksenov, Vasilii Davydov, and Aleksandr I. Panov. 从 minecraft 中的不完美演示看分层深度 Q 网络。 *Cogn. Cogn. Syst. Res.*, 65:74-78, 2021.
- [69] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. 通过世界模型掌握不同领域。 *ArXiv 预印本: Arxiv-2301.04104*, 2023.
- [70] Ryan Volum, Sudha Rao, Michael Xu, Gabriel DesGarennes, Chris Brockett, Benjamin Van Durme, Olivia Deng, Akanksha Malhotra 和 Bill Dolan. 打造一把铁剑通过对代码进行调整的大型语言模型提示来生成交互式游戏角色。 *第三届文字游戏大会论文集: 当语言遇到游戏》研讨会 (Wordplay 2022) 论文集*, 第25-43页, 美国西雅图, 2022年。计算语言学协会。
- [71] Haoqi Yuan, Chi Zhang, Hongcheng Wang, Feiyang Xie, Penglin Cai, Hao Dong, and Zongqing Lu. Plan4mc: *ArXiv preprint arXiv: 2303.16563*, 2023.

- [72] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, 伊娃-波特朗斯、克里斯托弗-波茨、阿迪蒂-拉古纳坦、罗布-赖希、任红玉、弗里达-荣、优素福-鲁哈尼、卡米洛-鲁伊斯、杰克-瑞安、克里斯托弗-雷、多尔萨-萨迪格、佐川诗织、凯沙夫-桑塔纳姆、安迪-施、克里希南-斯里尼瓦桑、亚历克斯-塔姆金、罗汉-陶里、阿明-W.Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. *ArXiv preprint arXiv: Arxiv-2108.07258*, 2021.
- [73] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 掌上电脑: *ArXiv preprint arXiv: Arxiv-2204.02311*, 2022.
- [74] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, 戴竹云、米拉克-苏兹贡、陈昕云、阿坎沙-乔德瑞、沙兰-纳兰、高拉夫-米什拉、亚当斯-于、文森特-赵、黄艳萍、戴宏坤、于、斯拉夫-彼得罗夫、埃德-H.Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models. *ArXiv preprint arXiv: Arxiv-2210.11416*, 2022.
- [75] Jiafei Duan, Samson Yu, Hui Li Tan, Hongyuan Zhu, and Cheston Tan. 具身人工智能概览: 从模拟器到研究任务》(A survey of embodied AI: from simulators to research tasks. *IEEE Trans. Emerg. Top. Comput. Intell.*, 6(2):230-244, 2022.
- [76] Dhruv Batra, Angel X. Chang, Sonia Chernova, Andrew J. Davison, Jia Deng, Vladlen Koltun, Sergey Levine, Jitendra Malik, Igor Mordatch, Roozbeh Mottaghi, Manolis Savva 和 Hao Su. 重排: *ArXiv preprint arXiv: Arxiv-2011.01975*, 2020.
- [77] Harish Ravichandar, Athanasios S Polydoros, Sonia Chernova, and Aude Billard. 机器人从演示中学习的最新进展》。《控制、机器人和自主系统年度综述》, 3:297-330, 2020 年。
- [78] 杰克-柯林斯、谢尔文-钱德、安东尼-范德科普和大卫-霍华德. 机器人应用物理模拟器综述》。 *IEEE Access*, 9: 51416-51431, 2021。
- [79] So Yeon Min, Devendra Singh Chaplot, Pradeep Ravikumar, Yonatan Bisk 和 R. Salakhutdinov. 电影: 用模块化方法遵循语言指令. *学习表征国际会议*, 2021 年。

- [80] Valts Blukis, Chris Paxton, Dieter Fox, Animesh Garg 和 Yoav Artzi. 用于高级自然语言指令执行的持久空间语义表征。 *第五届机器学习年会*, 2021年。
- [81] 瓦伦-奈尔、埃利奥特-舒马赫、杰弗里-曹、阿妮莎-卡南。Dera: 使用支持对话的解析代理增强大型语言模型补全。 *arXiv 预印本: Arxiv-2303.17071*, 2023。
- [82] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang 和 Michael S. Bernstein. Generative agents: *arXiv preprint arXiv: Arxiv-2304.03442*, 2023.
- [83] 吴越、Shrimai Prabhumoye、So Yeon Min、Yonatan Bisk、Ruslan Salakhutdinov、Amos Azaria、Tom Mitchell 和李远志。春天Gpt-4通过研究论文和推理超越RL算法。 *arXiv预印本arXiv: 2305.15486*, 2023.
- [84] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 用于程序合成的会话范例。 *ArXiv 预印本 arXiv: Arxiv-2203.13474*, 2022.
- [85] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese 和 Steven C. H. Hoi.Coder1: *ArXiv preprint arXiv: Arxiv-2207.01780*, 2022.
- [86] Xinyun Chen, Chang Liu, and Dawn Song. 执行引导的神经程序合成。 In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*.OpenReview.net, 2019.
- [87] Xinyun Chen, Dawn Song, and Yuandong Tian.神经程序合成的潜执行。 *arXiv preprint arXiv: Arxiv-2107.00101*, 2021.
- [88] Kevin Ellis, Maxwell I.Nye, Yewen Pu, Felix Sosa, Josh Tenenbaum, and Armando Solar-Lezama.编写、执行、评估: 使用 REPL 进行程序综合。 In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada, pages 9165-9174*, 2019年。
- [89] 李宇嘉、崔大伟、钟俊英、内特-库什曼、朱利安-施里特维泽、雷米-勒布隆德、汤姆-埃克斯、詹姆斯-基林、费利克斯-吉梅诺、奥古斯丁-达尔拉戈、托马斯-于贝尔、彼得-蔡、赛普里安-德-马松-德-奥图梅、伊戈尔-巴布施金、陈欣韵、黄柏森、约翰内斯-韦尔布、斯文-高瓦尔、阿列克谢-切雷帕诺夫、詹姆斯-莫洛伊、丹尼尔-J.Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals.用 alphacode 生成竞争级代码。 *ArXiv 预印本 arXiv: Arxiv-2203.07814*, 2022.
- [90] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman.训练验证器解决数学单词问题。 *ArXiv 预印本 arXiv: Arxiv- 2110.14168*, 2021.
- [91] Ansong Ni, Srini Iyer, Dragomir Radev, Ves Stoyanov, Wen tau Yih, Sida I. Wang 和 Xi Victoria Lin. Lever: *ArXiv preprint arXiv: Arxiv-2302.08468*, 2023.
- [92] Marta Skreta, Naruki Yoshikawa, Sebastian Arellano-Rubach, Zhi Ji, Lasse Bjørn Kristensen, Kourosh Darvish, Alán Aspuru-Guzik, Florian Shkurti 和 Animesh Garg.错误是有用的提示: *ArXiv preprint arXiv: Arxiv-2303.14100*, 2023.

A 方法

A.1 VOYAGER 算法

```
def voyager (
    环境 、                # 使用代码作为行动空间的环境 curriculum_agent 、        # 提出下一个任务的
    课程代理 action_agent 、    # 生成代码的行动代理
    critic_agent 、            # 用于自我验证的评论员代理 skill_manager 、        # 用于添加新
    技能和技能检索的技能管理器
):
    agent_state= environment . reset () while
    True :
        exploration_progress      =      ( curriculum_agent .
            get_exploration_progress (
                curriculum_agent . get_completed_tasks () ,
                curriculum_agent . get_failed_tasks () 、
            )
        )
        任务= curriculum_agent . propose_next_task ( agent_state ,
            exploration_progress
        )
        代码= 无 环境反馈= 无 执行错误= 无 评论=
        无
        success= False
        # 在转到下一个任务之前，最多尝试 4 轮 for i in range (4) :
            skills= skill_manager . retrieve_skills ( task ,
                environment_feedback
            )
            代码= action_agent . generate_code ( task 、
                code , environment_feedback ,
                execution_errors ,critique
            、
            技能 、
            )
            (
                agent_state ,
                environment_feedback ,
                execution_errors 、
            )= environment . step ( code )
            success , critique= critic_agent . check_task_success ( task ,
                agent_state
            )
            如果成功 :
                break
        if success :
            skill_manager . add_skill ( code ) curriculum_agent .
            add_completed_task ( task )
        否则 :
            add_failed_task ( 任务 )
```

伪代码 1: VOYAGER 算法。

A.2 提示

GPT-4 和 GPT-3.5 为用户提供了在三个选项中指定每条提示信息的角色的功能：

- 系统：系统：在整个对话过程中指导模型行为的高级指令。它为交互设定了总体基调和目标。
- 用户：指导助手立即做出下一步反应的详细指令。
- 助手：生成模型的响应信息。

详情请参见 <https://platform.openai.com/docs/guides/chat/introduction>。

为了节省令牌的使用，我们不再进行多轮对话，而是将系统提示和用户提示串联起来，以获得每个助手的回应。

A.3 自动课程

A.3.1 提示中的组件

GPT-4 的输入提示由几个部分组成：

- (1) 鼓励多样化行为的指令和施加限制的指令（以便提出的任务是可实现和可验证的）：完整的提示信息见第 A.3.4 节；
- (2) 代理的当前状态：
 - **库存**：带有计数的物品字典，例如 `{'cobblestone': 4, 'furnace': 1, 'stone_pickaxe': 1, 'oak_planks': 7, 'dirt': 6, 'wooden_pickaxe': 1, 'crafting_table': 1, '生铁': 14, '煤': 1}`；
 - **装备**：特工装备的盔甲或武器；
 - **附近街区**：距离特工 32 个街区范围内的一组街区名称，例如 "泥土"、"水"、"云杉板"、"草块"、"泥土小路"、"甘蔗"、"蕨类"；
 - **最近看到的其他图块**：不在附近或清单中的图块；
 - **附近的实体**：一组距离代理 32 个街区以内的实体名称，例如 "猪"、"猫"、"村民"、"僵尸"；
 - **代理看到的箱子列表**：箱子是代理可以存放物品的外部容器。如果某个箱子之前未被打开过，则其内容为 "未知"。否则，代理将看到每个箱子中的物品。
 - **生物群落**：例如，"平原"、"花林"、"草地"、"河流"、"海滩"、"森林"、"雪坡"、"冰冻山峰"、"古老的桦树林"、"海洋"、"阳光花原"、"石质海岸"；
 - **时间日出**、"白天"、"中午"、"日落"、"夜晚"、"午夜"中的一个；
 - **健康条和饥饿条**：最大值为 20；
 - **位置**代理在 Minecraft 世界中位置的 3D 坐标 (x, y, z) ；
- (3) 以前完成和失败的任务；
- (4) 其他背景：参见第 A.3.2 节；
- (5) 响应中的思维链提示 [46]：我们要求 GPT-4 首先推理当前的进度，然后建议下一个任务。

A.3.2 附加语境

我们利用 GPT-3.5 提出自我提问，以提供额外的语境。每个问题都与一个概念配对，该概念用于从维基知识库中检索最相关的文档[23]。我们将文档内容输入 GPT-3.5，用于自问自答。实际上，使用维基知识库是可选的，因为 GPT-3.5 已经很好地理解了 Minecraft 游戏机制。不过，如果 GPT-3.5 没有预先在特定领域接受过培训，那么外部知识库就会变得很有优势。完整提示见第 A.3.4 节。

A.3.3 热身时间表

在实践中，我们采用热身计划，根据代理已完成的任务数量，逐步将代理的状态和附加上下文纳入提示。这就确保了在探索过程中，提示会接触到越来越多的信息，因此

从基本技能开始，逐步过渡到更复杂、更多样的技能。我们在所有实验中使用的热身设置如表 A.1 所示。A.1.

表 A.1：自动课程的热身时间表。

提示信息	完成多少任务后
核心库存（仅包括原木、木板、木棒、工艺台、熔炉、泥土、煤炭、镐、剑和斧头）	0
设备附近的图块	0
位置附近的实体	0
完整库存	5
最近看到的其他积木 生物群落	7
健康条 饥饿	10
条 时间	10
其他背景	15
	15
	15
	15

A.3.4 完整提示

提示 1：用于自动课程的完整系统提示。问题-答案对列表代表额外的上下文。

你是一个有用的助手，告诉我下一个立即在威廉与做的任务。我的最终目标是发现尽可能多的不同的东西，完成尽可能多的不同的任务，并成为世界上最好的威廉与玩家。

我会给你以下信息：

问题 1: ...
 答案：...

问题 2: ...
 答案：...

问题 3: ...
 答案：...

...

生物群落：...

时间：...

附近区块：...

最近看到的其他区块：... 附近的实体（从最近到最远）：...健康：大于 15 表示我很健康 .饥饿：高于 15 表示我不饿 .位置：...

装备：如果我的库存中有更好的盔甲，你应该要求我装备它。

库存（xx / 36）：...

箱子：你可以要求我从这些箱子中存放或拿走物品。
 可能还有一些未知的箱子，你应该让我打开并检查未知箱子中的物品。

目前已完成任务：...

太难的失败任务：...

您必须遵循以下标准：

- 1) 您应该扮演导师的角色，根据我当前的学习进度指导我完成下一个任务。
- 2) 请非常具体地说明我需要收集什么资源，我需要制作什么，或我需要杀死什么小怪。

- 3) 下一个任务应该采用简洁的格式，例如" Mine [我的....."。
数量][块]"，" 工艺 [数量][项目]"熔炼 [数量][物品]"杀死 [数量][暴徒]"烹饪 [数量][食物]"，" 装备 [物品]" 等。装备 [物品]" 等。应该是一个短语。不要同时提出多个任务。不要提及任何其他内容。
- 4) 下一个任务不应该太难，因为我可能还没有必要的资源或学到足够的技能来完成它。
- 5) 下一个任务应该新颖有趣。我应该寻找稀有资源，使用更好的材料升级我的设备和工具，并发现新事物。我不应该重复做同样的事情。
- 6) 如果我需要收集更多的资源来完成更困难的任务，我有时可能需要重复某些任务。只有在必要时才重复任务。
- 7) 即使是在晚上，也不要让我建造或挖掘庇护所。我想探索世界，发现新事物。我不想呆在一个地方。
- 8) 应避免执行需要玩家状态以外的信息来验证的任务。例如，"放置 4 个火把"和"挖一个 2 x 1 x 2 的洞"就不太理想，因为它们需要屏幕的视觉确认。所有放置、建造、种植和交易任务都应避免。不要提出以这些关键字开头的任务。

您只能按照以下格式进行回复：回复格式：

推理：根据我上面列出的信息，推理下一个任务是什么。

任务：下一个任务。

下面是一个回答示例：

推理：现在库存是空的，砍倒一棵树获得一些木材。

任务：获取一根木头。

提示 2：用于提问的全系统提示。我们提供了好的和坏的示例，作为少量示例。

你是一个有用的助手，问的问题，以帮助我决定下一个立即在威廉与做的任务。我的最终目标是发现尽可能多的东西，完成尽可能多的任务，并成为世界上最好的威廉与玩家。

我会给你以下信息：

生物群落：...

时间：...

附近的块：...

最近看到的其他区块：... 附近的实体（从最近到最远）：...健康：

...

饥饿：...

位置：...

装备：... 库存（xx / 36）：...

箱子：...

目前已完成任务：...

太难的失败任务：...

您必须遵循以下标准：

- 1) 您应提出至少 5 个问题（但不超过 10 个问题），以帮助我决定下一个要立即完成的任务。每个问题后都应跟上问题所涉及的概念。
- 2) 您的问题应具体到威廉与中的一个概念。Bad example (the question is too general):

问题：玩 Minecraft 的最佳方式是什么？概念：未知

糟糕的示例（斧头仍然是通用的，您应该说明斧头的类型，如木制斧头）：

使用斧头采集资源有什么好处？概念：斧头

好例子：

问题：如何制作木镐？概念：木镐

3) 您的问题应该自成体系，不需要任何上下文。

坏例子（问题需要我当前生物群落的背景）：问题：我能在当前的生物群落中找到哪些积木？概念：未知

糟糕的例子（这个问题需要结合我目前的库存情况）：

问题：您目前最需要的资源是什么？概念：未知

错误示例（问题需要结合我当前的库存情况）：

问题：您有黄金或翡翠资源吗？概念：黄金

错误示例（问题需要结合我附近的实体）：

问题：你能看到附近有可以杀死作为食物的动物吗？

概念：食物

糟糕的例子（问题需要结合我附近的街区）：问题：附近有水源吗？

概念：水 好例子：

问题：我能在稀疏的丛林中找到哪些积木？

概念：稀疏的丛林

4) 不要问有关建筑任务的问题（例如建造一个避难所），因为这些任务对我来说太难了。

假设您当前的生物群落是稀疏的丛林。您可以提出这样的问题：

问题：我能在稀疏丛林中找到哪些物品？概念：稀疏丛林

问题：我可以在稀疏丛林中找到哪些小怪？概念：稀疏丛林

假设您看到附近有一只爬行者，而您以前从未战胜过爬行者。您可以提出这样的问题：

问题：如何打败爬山虎？概念：爬山虎

假设您上次完成的任务是 "制作一把木镐"。您可以提出这样的问题：

问题：制作完木镐后，我可以做哪些建议任务？

概念：木镐

下面是更多问题和概念示例：

问题：我可以在稀疏丛林中找到哪些矿石？

概念：稀疏丛林

(上面的概念不应该是 "矿石"，因为我需要查看 "稀疏丛林" 的页面才能知道在稀疏丛林里能找到什么矿石)。

问题：如何在稀疏丛林中获得食物？概念：稀疏丛林

(上面的概念不应该是 "食物", 因为我需要查找 "稀疏丛林 " 页面, 才能知道在稀疏丛林中可以找到哪些矿石)。

在 "人烟稀少的丛林 " 页面中查找在人烟稀少的丛林中可以获得哪些食物)

问题: 如何使用熔炉升级装备并制作有用的物品?

概念 : 熔炉

问题: 如何获得钻石矿石? 概念 : 钻石矿石

问题: 使用石镐比使用木镐有什么好处?

概念: 石镐

问题: 使用木板和木棍可以制作哪些工具?

概念: 木板

您只能按以下格式作答: 回答格式 :

推理 : ...

问题 1: ...

概念 1: ...

问题 2: ...

概念 2: ...

问题 3: ...

概念 3: ...

问题 4: ...

概念 4: ...

问题 5: ...

概念 5: ...

...

提示 3: 用于回答问题的完整系统提示。上下文代表维基知识库中的可选内容。

你是一个有用的助手, 回答了我关于威廉与魔兽的问题。

我会给你以下信息 :

问题 : ...

您将根据上下文 (仅在可用且有帮助的情况下) 和您自己对威廉与魔兽的了解来回答问题。

1) 请以 "答案: " 开始回答。

2) 如果您不知道答案, 请回答 " Answer : 未知 " 如果您不知道答案。

A.4 技能库

A.4.1 提示中的组件

GPT-4 的输入提示由以下部分组成:

- (1) 代码生成指南: 完整提示请参见第 A.4.2 节;
- (2) 由我们实现的控制原始 API: 这些 API 有双重用途: 它们演示了 Mineflayer API 的用法, 而且 GPT-4 可以直接调用它们。
 - `exploreUntil(bot, direction, maxTime= 60, callback)`: 允许代理在 `maxTime` 内沿固定方向探索。回调是代理执行的停止条件, 用于确定何时停止探索;
 - `mineBlock(bot, name, count= 1)`: 挖掘并收集指定数量的区块: 在 32 个街区的距离内开采并收集指定数量的街区;
 - `craftItem(bot, name, count= 1)`: 在附近的制作台制作物品;
 - `placeItem` (机器人, 名称, 位置): 在指定位置放置方块;
 - `smeltItem` (机器人, 物品名称, 燃料名称, 计数= 1): 用指定的燃料熔炼物品。附近必须有熔炉;

- `killMob(bot, mobName, timeout = 300)`: 攻击暴徒并收集其掉落的物品;
- `getItemFromChest(bot, 箱子位置, itemsToGet)`: 移动到指定位置的箱子, 并从箱子中获取物品;
- `CositItemIntoChest` (机器人、箱子位置、要存放的物品): 移动到指定位置的箱子, 并将物品存入箱子;

(3) 控制 Mineflayer 提供的原始 API:

- `await bot.pathfinder.goto(goal)`: 前往指定位置。请参阅下文了解如何设置目标;
- `new GoalNear(x, y, z, range)`: 将机器人移动到指定区块的指定范围内;
- `new GoalXZ(x, z)`: 用于没有特定 Y 水平的远距离目标;
- `new GoalGetToBlock(x,y,z)`: 不是进入区块, 而是直接靠近区块。这对于钓鱼、耕作、装桶和使用床都很有用;
- `new GoalFollow(entity, range)`: 在指定范围内跟随指定实体: 在指定范围内跟随指定实体;
- `new GoalPlaceBlock(position, bot.world, {})`: 定位机器人, 以便放置图块;
- `new GoalLookAtBlock(position, bot.world, {})`: 指向一个位置的路径, 在这个位置上可以看到位于 position 处的块的一个面;
- `bot.isABed(block)`: 如果 block 是一张床, 则返回 true;
- `bot.blockAt(position)`: 返回位置处的块;
- `await bot.equip(item, destination)`: 目的地必须是 "手"、"头"、"躯干"、"腿"、"脚" 和 "双手" 中的一个;
- `await bot.consume()`: 消耗机器人手中的物品。您必须先装备好要消耗的物品。适用于吃食物、喝药水等;
- `await bot.fish()`: 让机器人捕鱼。在调用此函数前, 必须先到达水块, 然后装备一根鱼竿。钓到鱼后, 机器人会自动停止钓鱼;
- `await bot.sleep(block)`: 一直睡到日出。您必须先到达床块;
- `await bot.activateBlock(block)`: 这与右键单击游戏中的区块相同。适用于按钮、门等。您必须先到达区块;
- `await bot.lookAt(position)`: 查看指定位置。在查看位置之前, 您必须先靠近该位置。要给水桶装水, 必须先查看水桶;
- `await bot.activateItem()`: 这与右击机器人手中的物品一样。在使用水桶等物品时非常有用。您必须先装备好要激活的物品;
- `await bot.useOn(entity)`: 这与右键单击游戏中的实体相同。用于剪羊毛。您必须先到达实体;

(4) 从技能库中获取技能;

(5) 上一轮生成的代码;

(6) 环境反馈: 提示中的聊天记录;

(7) 执行错误;

(8) 来自自我验证模块的批评;

(9) 代理的当前状态: 有关代理状态的各个要素, 请参见第 A.3.1 节;

(10) 自动课程提出的任务;

(11) 任务背景: 我们提示 GPT-3.5 征求关于如何解决任务的一般建议。在实践中, 这部分由自动课程处理, 因为它有一个系统的问题解答机制 (A.3.2); 节

(12) 思维链提示 [46] 作为回应: 我们要求 GPT-4 首先解释上一轮代码失败的原因, 然后给出完成任务的分步计划, 最后生成代码。完整提示见第 A.4.2 节。

A.4.2 完整提示

你是一个有用的助手，可以编写 Mineflayer javascript 代码来完成我指定的任何 Minecraft 任务。

下面是一些使用 Mineflayer API 编写的实用程序。

```
/*
探索直到找到铁矿石，使用 Vec3 (0, -1, 0) 因为铁矿石通常在地下
await explore Until ( bot, new Vec3 (0, -1, 0), 60, ()=> { const
  iron_ore= bot . find Block ({ 匹配 : mc Data .
    id , max Distance : 32 、
  });
  return iron_ore ;
});

探索直到找到一头猪，使用 Vec3 (1, 0, 1) 因为猪通常都在地表上
let pig= await explore Until ( bot, new Vec3 (1, 0, 1), 60, ()=> { const pig= bot .
  nearest Entity (( entity )=> {
    返回
    entity . name      ===      " pig "      && entity . position
    . distance To ( bot . entity . position )< 32
  });
  return pig ;
});
*/
async function explore Until ( bot , direction , max Time= 60 , callback ) {
  /*
  direction: Vec3, 只能包含 -1, 0 或 1 max Time: 数字, 探索的最长时间
  callback : 函数, 早期停止条件, 每秒调用一次, 如果返回值不为空, 探索将停止

  返回: 如果探索超时则为 null, 否则返回回调的返回值
  */
}

// Mine 3 cobblestone : mine Block ( bot , " stone ", 3); async
function mine Block ( bot , name , count= 1) {
  const blocks= bot . find Blocks ({ matching : (
    block ) => {
      return block . name=== name ;
    },
    max Distance : 32 , count :
    count 、
  });
  const targets= [];
  for ( let i= 0; i< Math . min ( blocks . length , count ); i ++ ) { targets . push ( bot .
    block At ( blocks [ i] ) );
  }
  await bot . collect Block . collect ( targets , { ignore No Path : true } );
}

// Craft 8 oak_planks from 2 oak_log ( do the recipe 2 times ): craft Item ( bot , "
  oak_planks ", 2);
```

提示 4: 生成代码的完整系统提示。

```

// 您必须在调用此函数前放置一个制作表 async function craft Item ( bot , name , count= 1) {
    const item= mc Data . items By Name [ name ]; const
    crafting Table= bot . find Block ({
        matching : mc Data . blocks By Name . crafting_table . id , max
        Distance : 32 、
    });
    await bot .
        new Goal Look At Block ( crafting Table . position , bot . world )
    );
    const recipe= bot . recipes For ( item . id , null , 1 , crafting Table ) [0]; await bot .
    craft ( recipe , count , crafting Table );
}

// 在玩家附近放置一个手工制作表, Vec3 ( 1 , 0 , 0 ) 只是一个示例, 你不应该总是使用它: place Item ( bot
, " crafting_table " , bot . entity . position . offset ( 1 , 0 , 0 ));
async 函数 place Item ( bot , name , position ) {
    const item= bot . inventory . find Inventory Item ( mc Data . items By Name [ name ]. id )
    ;
    // 查找参考块 const face Vectors = [
        new Vec3 ( 0 , 1 , 0 ) 、
        new Vec3 ( 0 , -1 , 0 ) 、
        new Vec3 ( 1 , 0 , 0 ) 、
        新 Vec3 ( -1 , 0 , 0 ) 、
        新 Vec3 ( 0 , 0 , 1 ) 、
        new Vec3 ( 0 , 0 , -1 ) 、
    ];
    let reference Block= null ; let face
    Vector= null ;
    for ( const vector of face Vectors ) {
        const block= bot . block At ( position . minus ( vector )); if (
        block ?. name !== " air " ) {
            reference Block= block ; face
            Vector= vector ; break ;
        }
    }
    // 您必须首先到达要放置的块位置, 等待 bot . pathfinder . goto ( new Goal Place Block (
    position , bot . world . { } ) ); // 您必须首先到达要放置的块位置, 等待 bot 、
    {} ) );
    // 您必须在调用放置区块之前装备物品 await bot . equip ( item , " hand " );
    await bot . place Block ( reference Block , face Vector );
}

// 使用 1 块橡木板作为燃料, 将 1 块生铁冶炼成 1 块铁锭 : smelt Item ( bot , " raw_iron " , "
oak_planks " );
// 在调用此函数之前, 必须先放置一个熔炉
async function smelt Item ( bot , itemName , fuelName , count= 1) { const item= mc
Data . items By Name [ item Name ];
const fuel= mc Data . items By Name [ fuel Name ]; const
furnace Block= bot . find Block ({ 匹配 : mc Data .
    match : mc Data . blocks By Name . furnace . id , max
    Distance : 32 、
});
await bot .
    new Goal Look At Block ( furnace Block . position , bot . world )
);
const furnace= await bot . open Furnace ( furnace Block ); for ( let i=
0; i< count ; i ++ ) {
    await furnace . put Fuel ( fuel . id , null , 1);
}

```



```

        await furnace . put Input ( item . id , null , 1);
        // await furnace ;
        await furnace ;
    }
    await furnace . close ();
}

// 杀死一头猪并收集掉落物品: kill Mob ( bot , " pig " , 300) ; async function kill Mob ( bot ,
mobName , timeout= 300) {
    const entity= bot . nearest Entity ( (
        entity ) =>
            entity . name      ===      mob Name      && entity .
            position . distance To ( bot . entity . position )< 32
    );
    实体.位置.x, 实体.位置.y, 实体.位置.z 。
    新目标块 ( 实体 . 位置 . x, 实体 . 位置 . y, 实体 位置 . z)
    );
}

// Get a torch from chest at (30 , 65 , 100) : get Item From Chest ( bot , new Vec3 (30 , 65 ,
100) , {" torch " : 1});
// 无论机器人离箱子有多远, 这个函数都会工作
.
async function get Item From Chest ( bot , chest Position , items To Get ) { await move
To Chest ( bot , chest Position );
const chest Block= bot . block At ( chest Position ); const chest=
await bot . open Container ( chest Block ); for ( const name in
items To Get ) {
    const item By Name= mc Data . items By Name [ name ];
    const item= chest . find Container Item ( item By Name . id); await chest
    . withdraw ( item . type , null , items To Get [ name ]);
}
    await close Chest ( bot , chest Block )
}

// 在 (30 , 65 , 100) 处向箱子中存入一个火炬 : deposit Item Into Chest ( bot , new Vec3 (30 , 65 ,
100) , {" torch " : 1});
// 无论机器人离箱子有多远, 这个函数都会工作
.
async function deposit Item Into Chest ( bot , chest Position , items To Deposit ) { // 无论机器人离
箱子有多远, 这个函数都会工作。
    ) {
        await move To Chest ( bot , chest Position );
        const chest Block= bot . block At ( chest Position ); const chest=
        await bot . open Container ( chest Block ); for ( const name in
        items To Deposit ) {
            const item By Name= mc Data . items By Name [ name ];
            const item= bot . inventory . find Inventory Item ( item By Name . id); await
            chest . deposit ( item . type , null , items To Deposit [ name ]);
        }
        await close Chest ( bot , chest Block );
    }
}

// Check the items inside the chest at (30 , 65 , 100) : check Item Inside
Chest ( bot , new Vec3 (30 , 65 , 100) );
// 您只需调用此函数一次, 无需任何操作即可完成检查箱子内物品的任务。
async function check Item Inside Chest ( bot , chest Position ) { await move
To Chest ( bot , chest Position );
const chest Block= bot . block At ( chest Position ); await bot .
open Container ( chest Block );
// 如果要求您打开箱子, 您必须在打开箱子后关闭箱子

```

```

    await close Chest ( bot , chest Block );
}

await bot .这个函数可以改变你的主要装备。
// 下面是一些你可以使用的目标:
new Goal Near ( x, y, z, range ); // Move the bot to a block within the specified range of the
    specified block .x"、"y"、"z 和 "范围 "都是 "数字"。
new Goal XZ ( x, z); // 对于没有特定 Y 值的 远距离目标非常有用。x'和'z'是'数字'。
new Goal Get To Block ( x, y, z); // 并非进入区块,而是直接靠近它。对捕鱼、耕作、装桶和铺床很有用
    。x"、"y "和 "z "都是 "数字"。
new Goal Follow ( entity , range ); // 在指定范围内跟随指定实体。实体 '是'实体', '范围'是'数字'
new Goal Place Block ( position , bot . world , {}); // 定位机器人,以便放置图块。位置'是'Vec3
new Goal Look At Block ( position , bot . world , {}); // 路径进入一个位置,在该位置上的块面是可
    见的。位置'是'Vec3

//          其他      Mineflyer      函数          可以使用: bot . is ABed ( bed Block );
// 如果'bedBlock'是床,则返回 true bot . block At ( position ); // 返回'position'处的块。位置
    是 'Vec3

// 这些是您可以使用的其他 Mineflyer 异步函数 :
await bot . equip ( item , destination ); // 在指定的目的地装备物品。项目 "是 "物品", "目的地 "只能是
    "手"、"头"、"躯干"、"腿"、"脚"、"非手"。
您必须先装备要消耗的物品。适用于吃食物,喝药水等。
await bot . fish (); // 让机器人捕鱼。调用此函数之前,您必须先到达一个水块,然后装备一根鱼竿。钓到鱼后
    , 机器人会自动停止钓鱼
await bot . sleep ( bed Block ); // Sleep until sunrise.您必须先到达床块
await bot . activate Block ( block ); // 这等同于在游戏中右击一个区块 .对于按钮、门、使用锄头等都很
    有用。
    您必须先到达床块
await bot . look At ( position ); // 查看指定位置。在查看位置之前,必须先靠近该位置。要将水桶装满
    水,必须先查看 At。"position "是 "Vec3"。
await bot . activate Item (); // 这与右键点击使用机器人手中的物品相同。对于使用水桶等物品非常有用。您
    必须先装备好要激活的物品
await bot . use On ( entity ); // 这与右键点击游戏中的实体相同。用于剪羊毛、装备马具
    等。您必须先进入实体

{ retrieved_skills }

在每一轮对话中,我都会给出上一轮对话的代码 : ...
执行错误 : ... 聊天记录 : ...
生物群落 : ...
时间 : ...
附近区块 : ...
附近的实体 (从最近到最远): 健康 : ...

```

饥饿 : ...
位置 : ...
装备 : ... 库存 (xx /36) : ...
箱子 : ...
任务 : ...
背景 : ...
评论 : ...

然后, 您应给我答复

解释 (如适用): 您的计划中是否缺少任何步骤? 为什么代码不能完成任务? 聊天记录和执行错误意味着什么?

计划: 如何逐步完成任务。您应该关注清单, 因为它告诉您有什么。任务完整性检查也基于您的最终库存。

代码:

- 1) 编写一个以机器人为唯一参数的异步函数。
- 2) 尽可能重复使用上述有用的程序。
 - 使用 "挖掘区块 (机器人、名称、数量)" 来收集区块。不要直接使用 "bot .dig"。
 - 使用 'craft Item (bot , name , count)' 制作品物。不要直接使用 '机器人'。
 - 使用 "熔炼物品 (bot , name count)" 熔炼物品。不要直接使用 "bot .open Furnace"。
 - 使用 '放置物品 (bot , name , position)' 放置图块。不要直接使用 '机器人'。
 - 使用 'kill Mob (bot , name , timeout)' 来杀死小怪。不要直接使用 '机器人'。
- 3) 您的函数将被重复用于构建更复杂的函数。因此, 您应该使其具有通用性和可重用性。你不应该对库存做出强烈的假设 (因为它可能会在以后发生变化), 因此在使用它们之前, 你应该总是检查你是否拥有所需的项目。如果没有、
如果没有, 则应首先收集所需物品, 然后重复使用上述有用的程序。
- 4) 上一轮代码 "部分中的函数将不会被保存或执行。请勿重复使用此处列出的函数。
- 5) 在函数之外定义的任何内容都将被忽略, 请在函数中定义所有变量。
- 6) 调用 "bot . chat "来显示中间进度。
- 7) 在找不到东西时, 使用 "探索直到 (机器人, 方向, 最大距离, 回调)"。在开采地块或杀死小怪之前, 你应该经常调用它。您应该每次随机选择一个方向, 而不是一直使用 (1, 0, 1)。
- 8) 对于 "机器人.找块" 和 "机器人.找块", "最大距离" 应始终为 32。
查找区块 "的" "最大距离" 应始终为 32。不要作弊。
- 9) 请勿编写无限循环或递归函数。
- 10) 不要使用 "bot . on "或 "bot . once "来注册事件监听器。您绝对不需要它们。
- 11) 以有意义的方式命名您的函数 (可以从名称推断任务)。

您只能使用下面描述的格式进行响应: 回答格式:

解释: ... 计划:

- 1) ...
- 2) ...
- 3) ...

...

代码:

javascript

// 辅助函数 (仅在需要时使用, 尽量避免使用)

...

```
// 主函数在辅助函数之后
async function your Main Function Name ( bot ) {
  // ...
}
```

提示 5：用于生成函数描述的完整系统提示。该提示用于向技能库中添加新技能。我们在提示中举了一个例子。

您是一名乐于助人的助手，您将用 `Mineflayer-javascript` 代码编写给定函数的说明。

- 1) 请勿提及函数名称。
- 2) 不要提及任何有关 "机器人聊天 "或辅助函数的内容。
- 3) 在主函数之前可能会有有一些辅助函数，但您只需描述主函数即可。
- 4) 尽量用不超过 6 句话概括该功能。
- 5) 您的回复应为一行文本。

例如，如果函数是

```
async function mine Cobblestone ( bot ) { // Check if wooden pickaxe is in the inventory , if not , craft one.
  // 检查库存中是否有木镐，如果没有，则制作一个
  let wooden Pickaxe= bot . inventory . find Inventory Item ( mc Data . items By Name [" wooden_pickaxe "]. id);
  如果 (≥ wooden Pickaxe ) {
    bot . chat ( " Crafting a wooden pickaxe ." ) ; await
    craft Wooden Pickaxe ( bot );
    木镐= bot . inventory . find Inventory Item ( mc Data . items By Name [" wooden_pickaxe "]. id);
  }

  // 如果木镐存在，则装备木镐。
  await bot . equip ( wooden Pickaxe , " hand " );

  // 探索，直到找到一块石块
  await explore Until ( bot , new Vec3 ( 1 , -1 , 1 ) , 60 , ()=> { const stone= bot . find Block ( { id , max Distance : 32 } );
    if ( stone ) {
      return true ;
    }
  } );

  // 开采 8 块鹅卵石 块 使用 木制 镐 bot . chat ( " Found a stone block . Mining 8 cobblestone blocks ." ) ; await mine Block ( bot , " stone " , 8);
  bot . chat ( " Successfully mined 8 cobblestone blocks ." ) ;

  // 保存开采 8 块鹅卵石的事件 bot . save ( " cobblestone_mined " );
} else {
  bot . chat ( " Failed to craft a wooden pickaxe . Cannot mine cobblestone ." ) ;
}
}
```

主函数是 "开采鹅卵石"。然后你可以写：

该功能是使用木镐开采 8 块鹅卵石。

首先检查库存中是否有木镐。如果没有，则制作一把。如果有木镐，则将木镐装备在手中。然后，探索环境，直到找到石块。找到石块后，使用木镐开采共 8 块鹅卵石。

A.4.3 示例

```
async 函数 手工制作木板 ( bot ) {
  const log Names= [ " oak_log ", " birch_log ", " spruce_log ", " jungle_log ".
    , " acacia_log ", " dark_oak_log ", " mangrove_log "];
  const plank Names = [ " oak_planks ", " birch_planks ", " spruce_planks ", " jungle_planks
    ", " acacia_planks ", " dark_oak_planks ", " mangrove_planks "];
  const log In Inventory= log Names . find ( log Name=> bot . inventory . count ( mc Data .
    items By Name [ log Name ]. id)> 0);
  如果 (≥ log Inventory ) {
    bot . chat ( " No wooden log in inventory . Mining a wooden log ... " ); await mine Wood
    Log ( bot );
  }
  const log Index= log Names . index Of ( log In inventory ); const
  plank Name = plank Names [ log Index ]; bot . chat ( '
  Crafting 4 ${ plank Name }... ' );
  await craft Item ( bot , plank Name , 1); bot .
  chat ( ' 4 ${ plank Name } crafted . ' );
}
```

技能库示例 1: 制作木板。

```
async 函数 mine Ten Cobbled Deepslate Below Y 0 ( bot ) {
  // 装备铁镐
  const iron Pickaxe= bot . inventory . find Inventory Item ( mc Data . items By Name [ "
    iron_pickaxe " ]. id );
  await bot . equip ( iron Pickaxe , " hand " );

  // 查找 Y=0 以下的鹅卵石深板块
  const cobbled Deepslate Blocks= await explore Until ( bot , new Vec3 ( 1 、
    -1 , 1 ) , 60 , ()=> {
    const cobbled Deepslate= bot . find Block ({
      block By Name [ " cobbled_deepslate " ]. id , max Distance : 32 、
      position : pos=> pos . y< 0
    });
    return cobbled Deepslate ;
  });

  // 开采 10 个鹅卵石深层板块
  await mine Block ( bot , " cobbled_deepslate " , 10 ) ; bot . chat ( "
  10 cobbled_deepslate mined below Y =0. " );
}
```

技能库示例 2: MineTenCobbledDeepslateBelowY0。

```
async 函数 find Suitable Position ( bot ) {  
  const offsets= [ new Vec3 (1 , 0 , 0) , new Vec3 (-1 , 0 , 0) , new Vec3 (0 ,  
    0 , 1) , new Vec3 (0 , 0 , -1) ];  
  for ( const offset of offsets ) {  
    const position= bot . entity . position . offset ( offset . x, offset . y, offset . z);  
    const block= bot ;  
  }  
}
```

技能库示例 3: smeltFiveRawIronV2.

```

        if ( block . name=== " air " ) {
            返回 position ;
        }
    }
    return null ;
}

async function smelt Five Raw Iron ( bot ) {
    // 检查库存中是否有熔炉
    const furnace Item= bot . inventory . find Inventory Item ( mc Data . items By Name
        . furnace . id);

    // 如果没有，则使用可用的鹅卵石制作一个熔炉。
    await craft Furnace ( bot )
}

// 找到合适的位置放置熔炉
const furnace Position= await find Suitable Position ( bot ); if (炉子位置 ) {
    bot . chat ( " Could not find a suitable position to place the furnace . " );
    返回 ;
}

// 将熔炉放置在合适的位置 await place Item ( bot , " furnace ",
furnace Position );

// 使用现有煤炭作为燃料冶炼 5 块生铁 await smelt Item ( bot , "
raw_iron ", " coal ", 5); bot . chat ( " 5 块生铁冶炼完成 . " );
}

```

技能库示例 4: fillBucketWithWater.

```
async function fill Bucket With Water ( bot ) {  
  // 查找附近的水块  
  const water Block= await explore Until ( bot , new Vec3 (1 , 0 , 1) , 60 , ()  
    =>{  
      const water= bot .  
        matching : mc Data . blocks By Name . water . id , max  
        Distance : 32  
    });  
  返回 water ;  
});  
如果 (水块)  
  bot . chat ( " Could not find water ." ); return ;  
}  
  
// 到达水块  
const adjacent Block= water Block . position . offset (0 , 1 , 0); await bot .  
pathfinder . goto ( new Goal Get To Block ( adjacent Block . x, adjacent Block . y, adjacent  
  Block . z)  
);  
  
// 查看水块  
await bot . look At ( water Block . position , true );  
  
// 装备水桶  
const bucket= bot . inventory . find Inventory Item ( mc Data . items By Name . bucket . id)  
;  
await bot . equip ( bucket , " hand " );
```



```

// 激活水桶以收集水
await bot ;
bot . chat ( " Filled the bucket with water ." );
}

```

```

async 函数 catch Five Fish Safely ( bot ) {
  // 检查机器人的库存中是否有鱼竿
  let fishing Rod= bot . inventory . find Inventory Item ( mc Data . items By Name .
    fishing_rod . id );
  如果 (估计钓鱼竿 ) {
    await craft Fishing Rod ( bot );
    钓鱼竿= bot . inventory . find Inventory Item ( mc Data . items By Name . fishing_rod .
      id );
  }

  // Find a nearby water block let water
  Block ;
  while (估计水块 ) {
    水块= await explore Until ( bot , new Vec3 ( 1 , 0 , 1 ) , 60 , ( ) =>
    {
      const found Water Block= bot . find Block ( { matching :
        mc Data . blocks By Name . water . id , max Distance :
        32
      } );
      return found Water Block ;
    } );
    如果 (水块)
      bot . chat ( " No path to the water block . Trying to find another water block ... " )
    ;
  }
}

// 移动到与水块相邻的区块
const adjacent Block= water Block . position . offset ( 0 , 1 , 0 ); await bot .
  pathfinder . goto ( new Goal Block ( adjacent Block . x . , adjacent Block . z ));
} // Move to a block adjacent to the water block、
  x , adjacent Block . y , adjacent Block . z ));

// 查看水块
await bot . look At ( water Block . position );

// 装备钓竿
await bot . equip ( fishing Rod , " hand " );

// 在水中捕鱼 5 次 for ( let i= 0; i< 5; i
++) {
  try {
    await bot ;
    bot . chat ( ' Fish ${ i+ 1} caught . ' );
  } catch ( 错误 ) {
    if ( error message === " 钓鱼 已取消 " ) { bot . chat ( " 钓鱼
      已取消。再试一次..." ); i - -; // 重试相同的迭代次数
    } else {
      throw error ;
    }
  }
}
}
}

```

技能库示例 5: catchFiveFishSafely.

A.5 自我验证

A.5.1 提示符中的组件

GPT-4 的输入提示由以下部分组成：

- (1) 代理的状态：我们将最近看到的其他区块和附近的实体排除在代理状态之外，因为它们对评估任务的完成度并无用处。有关代理状态的各个要素，请参见第 A.3.1 节；
- (2) 自动课程提出的任务；
- (3) 任务背景：我们提示 GPT-3.5 征求关于如何解决任务的一般建议。实际上，这部分内容由自动课程处理，因为它有一个系统的问题解答机制（A.3.2）；节
- (4) 思维链提示 [46] 作为回应：我们要求 GPT-4 首先对任务的成败进行推理，然后输出一个布尔变量表示任务的结果，最后在任务失败时向代理提出批评。
- (5) 语境中学习的少量示例 [36-38]。

A.5.2 完整提示

提示 6：用于自我验证的完整系统提示。

你是一个助理，评估我玩威廉与进展情况，并提供有用的指导。

您需要评估我是否达到了任务要求。

超过任务要求也被视为成功，而未达到任务要求则需要您提供批评意见以帮助我改进。

我将向您提供以下信息：

生物群落：任务执行后的生物群落。时间：当前时间。

附近的区块：周围的区块。这些积木还未收集。不过，这对一些放置或种植任务很有用。

健康：我当前的健康状况。

饥饿度：我当前的饥饿度。对于进食任务，如果我的饥饿值是 20.0，那么我成功吃到了食物。

位置：我当前的位置。

装备：我的最终装备。对于制作任务，我有时会装备制作好的物品。

库存（xx / 36）：我的最终库存。对于采矿和冶炼任务，你只需要查看库存。

箱子（Chests）：如果任务要求我将物品放入箱子，您可以在这里找到箱子信息。

任务：我需要完成的目标。Context（上下文）：任务的上下文。

您只能以下面描述的 JSON 格式回复：

```
{
  "reasoning": "reasoning",
  "success": boolean,
  "critique": "critique",
}
```

确保响应可被 Python 'json.loads' 解析，例如 无尾部逗号、无单引号等。

下面是一些示例：

输入：

Inventory (2 / 36) : { ' oak_log ': 2 , ' spruce_log ': 2 }

任务: 开采 3 根原木

响应 :

```
{
  "推理": " 你需要开采 3 根原木。您有 2 根橡木原木和 2 根云杉原木, 加起来就是 4 根原木。",
  "success ": true, "
  critique ": ""
}
```

输入 :

库存 (3 / 36) : { ' crafting_table ': 1 , ' spruce_planks ': 6 , ' stick ': 4 }

任务: 制作一把木镐 响应 :

```
{
  "推理": "你有足够的材料来制作一把木镐, 但你没有制作它"。 ,
  "成功 ": false 、
  "批判": " 使用 3 块云杉木板和 2 根木棍, 用制作台制作一把木镐。"
}
```

输入 :

库存 (2 / 36) : { ' 生铁 ': 5 , ' 石镐 ': 1 }

任务: 开采 5 块铁矿石 响应:

```
{
  "在威廉与魔兽中开采铁矿石会获得生铁。您的库存中有 5 个生铁" . ,
  "success ": true, "
  critique ": ""
}
```

输入 :

生物群落: 平原

附近的块 : 石头 , 泥土 , 草块 , 草地 , 农田 , 小麦 库存 (26 / 36) : ...

任务 : 种植 1 粒小麦种子 .

响应 :

```
{
  "理由是: "对于种植任务, 库存信息是无用的。
  .在附近的区块中, 有农田和小麦, 这意味着你成功地种植了小麦种子"。 ,
  "success ": true, "
  critique ": ""
}
```

输入 :

库存 (11 / 36) : { ...rotten_flesh ': 1 }

任务: 杀死 1 个僵尸 上下文: ...

响应

```
{
```

```

"推理": " 您的库存中有腐肉，这意味着
您成功杀死了一只僵尸"。", " 成功 ": true 、
"评论": ""
}

输入：
饥饿: 20.0 / 20.0

库存 ( 11 / 36 ) : ... 任务：吃

1 ...

情境：...

响应
{
  "推理": " 对于所有进食任务，如果玩家的饥饿值是 20.0，那么玩家成功吃掉了食物"。",
  "成功": true, "批判":
  ""
}

输入：
附近的方块：箱子

库存 ( 28 / 36 ) : { ' 铁轨 ': 1, ' 煤 ': 2, ' 橡木板 ': 13, ' 铜块 ': 1, ' 闪长岩 ': 7, ' 熟牛肉 ': 4, ' 花
岗岩 ': 22, ' 花岗岩 ': 4。
  copper_block ': 1, ' diorite ': 7, ' cooked_beef ': 4, ' granite ': 22, ' 鹅卵石_deepslate':
  23, ' feather ': 4
  鹅卵石_deepslate": 23, ' feather": 4, ' leather": 2, ' cook_icken
  cooked_chicken ': 3, ' white_wool ': 2, ' stick ': 3, ' black_wool ': 1、
  ' stone_sword ': 2, ' stone_hoe ': 1, ' stone_axe ': 2, ' stone_shovel ':
  2, ' cooked_mutton ': 4, ' cobblestone_wall ': 18, ' crafting_table ':
  1, ' 熔炉 ': 1, ' 铁镐 ': 1, ' 石镐 ': 1, ' 生铜 ': 12
  生铜 ': 12}

箱子：
(81, 131, 16) : { ' 安山岩 ': 2, ' 泥土 ': 2, ' 鹅卵石 ': 75, ' 石块 ': 1
  wooden_pickaxe ': 1, ' wooden_sword ': 1}

任务： 将无用物品放入位于 (81, 131, 16) 的箱子中 上下文： ...

响应
{
  "推理": "放入无用物品后，您的库存中有 28 件物品。
超过了 20 件。您需要将更多物品从您的库存中存入箱子"。",
  "成功 ": false 、
  "批判": " 存放更多无用物品，例如铜块、闪长岩、花岗岩、鹅卵石_深石板、羽毛和皮革，以满足库存中只有
20 个可用插槽的要求。
  ."
}

```

A.6 VOYAGER 与先前作品的系统级比较

我们在表 A.2 中进行了系统级比较。A.2.Voyager 是唯一结合了自动课程、迭代规划和技能库的方法。此外，它无需任何梯度更新就能学会玩 Minecraft。

	VPT [8]	DreamerV3 [69]	DECKARD [53]	DEPS [55]	Plan4MC [71]	航海家
演示	视频	无	视频	无	无	无
奖励	稀疏	密集	稀疏	无	密集	无
观测	仅像素	像素和元	像素与库存	反馈与库存	像素与元	反馈与元与库存
操作	键盘与鼠标	离散	键盘和鼠标	键盘和鼠标	分立式	代码
自动课程			✓			✓ (上下文 GPT-4 建议提议)
迭代计划计划				✓		✓ (3 种反馈)
技能库					✓ (预定义)	✓ (自我生成)
无梯度						✓

表 A.2: VOYAGER 与之前研究成果的系统级比较。

B 实验

B.1 实验设置

我们的模拟环境基于 MineDojo [23]，并使用 Mineflayer [52] JavaScript API 进行电机控制（第 A.4.2）。节此外，我们还在 Mineflayer 函数中加入了许多 `bot.chat()`，以提供丰富的环境反馈，并通过 `try-catch` 异常实现各种条件检查，以保证连续执行。如果机器人死亡，它会在最近的地面附近复活，并保留其库存，以便进行不间断的探索。程序执行后，机器人会回收其制作台和熔炉。有关详细实现，请参阅我们的代码库。

B.2 基准线

ReAct [29] 使用思维链提示 [46]，通过 LLM 生成推理轨迹和行动计划。我们将环境反馈和代理状态作为观察结果提供给它。ReAct 会从头开始生成一轮代码，然后进行三轮代码完善。然后重复这一过程，直到达到最大提示迭代次数。

Reflexion [30] 建立在 ReAct [29] 的基础上，通过自我反思来推断更直观的未来行动。我们为其提供环境反馈、代理状态、执行错误和自我验证模块。与 ReAct 类似，Reflexion 也要经历一轮从零开始的代码生成，然后是三轮代码完善。然后重复这一过程，直到达到最大提示迭代次数。

AutoGPT [28] 是一种流行的软件工具，它通过将高级目标分解为多个子目标，并在 ReAct 式循环中执行这些子目标，从而实现 NLP 任务的自动化。我们通过使用 GPT-4 进行任务分解来重新实现 AutoGPT，并为其提供代理状态、环境反馈和执行错误作为子目标执行的观察结果。与 VOYAGER 相比，AutoGPT 缺少用于积累知识的技能库、用于评估任务成功与否的自我验证以及用于开放式探索的自动课程。在每个子目标的执行过程中，如果没有执行错误发生，我们就认为该子目标已经完成，并继续执行下一个子目标。否则，我们会完善程序，直到完成三轮代码完善（相当于四轮代码生成），然后进入下一个子目标。如果连续三个子目标都没有获得新项目，我们就会通过重新运行任务分解来重新计划。

所有基线的任务都是 "探索世界并获取尽可能多的物品"。

表 A.3: VOYAGER 与基线的比较。

	ReAct [29]	Reflexion [30]	AutoGPT [28]	旅行者
思维链 [46]	✓	✓	✓	✓
自我验证		✓		✓
环境反馈	✓	✓	✓	✓
执行错误		✓	✓	✓
代理状态	✓	✓	✓	✓
技能库				✓
自动课程				✓

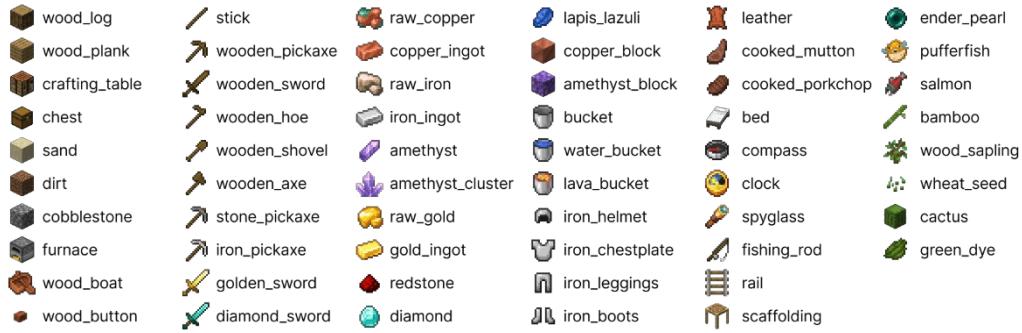


图 A.1: 带有相应名称的 Minecraft 项目图标。

B.3 消融

我们在 VOYAGER 中消融了 6 种设计选择（自动课程、技能库、环境反馈、执行错误、自我验证和用于代码生成的 GPT-4），并研究了它们对探索性能的影响。

- **手动课程**: 我们用手动设计的钻石开采课程替代自动课程: "开采 3 根原木"、"制作 1 个制作台"、"制作 1 把木镐"、"开采 11 块鹅卵石"、"制作 1 把石镐"、"制作 1 个熔炉"、"开采 3 块铁矿石"、"熔炼 3 块铁矿石"、"制作 1 把铁镐"、"开采 1 颗钻石"。手动课程需要人工设计, 无法扩展到开放式探索中。
- **随机课程**: 我们对 VOYAGER 获得的 101 件物品进行整理, 并通过随机选择一件物品作为下一个任务来创建随机课程。
- **无技能库**: 我们移除技能库, 消除代码生成中的技能检索。
- **无环境反馈**: 我们将环境反馈（聊天记录）排除在代码生成提示之外。
- **无执行错误**: 我们在生成代码的提示中排除了执行错误。
- **无自我验证**: 对于每项任务, 我们在不进行自我验证的情况下生成代码, 并对程序进行 3 轮改进（相当于总共生成 4 轮代码）。
- **GPT-3.5**: 我们用 GPT-3.5 取代 GPT-4, 用于代码生成。自动课程和自我验证模块保留 GPT-4。

B.4 评估结果

B.4.1 探索性显著提高

图 1 中每个图标的含义如图 A.1 所示。

我们对每种方法进行了三次试验。VOYAGER 在每次试验中收集到的项目为

- **试验 1:** 铁砧木"、"石铲"、"铁绑腿"、"鱼竿"、"河豚"、"橡木原木"、"熟羊肉"、"绿色染料"、"燧石"、"箱子"、"铁剑"、"绳子"、"珍珠"、"生铜"、"制作台"、仙人掌"、"青金石"、"铁镐"、"镐头"、"石镐"、"木蹄"、"脚手架"、"木棍"、"猪排"、"镐头块"、"砾石"、"草块"、"白床"、"骨头"、"泥土"、"羊肉"、"白羊毛"、"橡木树皮"、"煤"、"竹子"、"木镐"、"腐肉"、"熟猪排"、"鳕鱼"、"铁靴"、"避雷针"、"闪长岩"、"水桶"、"剪刀"、"熔炉"、"安山岩"、"花岗岩"、水桶"、"木剑"、"砂岩"、"铁头盔"、"生铁"、"沙子"、"槐木"、"熟鳕鱼"、"橡木板"、"蔚蓝蓝"、"铁铲"、"槐木板"、"盾牌"、"铁斧"、"铁胸甲"、"鹅卵石"；
- **试验 2:** 铁砧子"、"凝灰岩"、"石铲"、"铁绑腿"、"鱼竿"、"熟羊肉"、"云杉板"、"火药"、"紫水晶碎片"、"箱子"、"绳子"、"熟鲑鱼"、"铁剑"、"生铜"、制作台"、"火把"、"青金石"、"铁镐"、"镐头"、"石镐"、"木蹄"、"木棍"、"紫水晶块"、"三文鱼"、"卡路里"、"砾石"、"白床"、"骨头"、"泥土"、羊肉"、"白羊毛"、"望远镜"、"煤"、"木镐"、"鳕鱼"、"铁靴"、"百合花垫"、"鹅卵石深石板"、"闪电棒"、"雪球"、"石斧"、"光滑玄武岩"、"闪长岩"、"水桶"、熔炉"、"安山岩"、"水桶"、"花岗岩"、"盾牌"、"铁盔"、"生铁"、"鹅卵石"、"云杉木"、"熟鳕鱼"、"绊线钩"、"石锄"、"铁胸甲"、"石剑"；
- **试验 3:** 云杉板"、"泥土"、"盾牌"、"红石"、"钟"、"钻石剑"、"铁制胸牌"、"石镐"、"皮革"、"绳子"、"鸡"、"箱子"、"闪长岩"、"铁制紧身裤"、"黑羊毛"、"鹅卵石墙"、"鹅卵石"、"熟鸡"、"羽毛"、"石剑"、"生金"、"砾石"、"桦木板"、"煤"、"鹅卵石深石板"、"橡木板"、"铁镐"、"花岗岩"、"凝灰岩"、"制作台"、"铁头盔"、"石蹄"、铁镐头"、"石斧"、"桦木船"、"木棍"、"沙子"、"骨头"、"生铁"、"牛肉"、"铁轨"、"橡木树苗"、"海带"、"金镐头"、"桦木原木"、"小麦种子"、"熟羊肉"、"熔炉"、"箭"、"石铲"、白羊毛"、"安山岩"、"丛林岩板"、"羊肉"、"铁剑"、"铜砧"、"钻石"、"火炬"、"橡木原木"、"熟牛肉"、"铜块"、"火石"、"骨粉"、"生铜"、"木镐"、"铁靴"、"木剑"。

ReAct [29] 在每个试验中收集的物品是

- **试验 1:** "竹子"、"泥土"、"沙子"、"小麦种子"；
- **试验 2:** "泥土"、"兔子"、"云杉木"、"云杉树苗"；
- **试验 3:** "土"、"尖滴石"；

Reflexion [30] 在每个试验中收集的物品是

- **试验 1:** "制作台"、"橙色郁金香"、"橡木板"、"橡木原木"、"泥土"；
- **试验 2:** "云杉木"、"泥土"、"粘土球"、"沙子"、"碎石"；
- **试验 3:** "小麦种子"、"橡木原木"、"泥土"、"桦木原木"、"沙子"。

AutoGPT [28] 在每个试验中收集的项目是

- **试验 1:** "羽毛"、"橡木原木"、"皮革"、"棍子"、"猪排"、"鸡"、"制作台"、"小麦种子"、"橡木板"、"泥土"、"羊肉"；
- **试验 2:** 木镐"、"铁砧板"、"石头"、"煤炭"、"云杉板"、"绳子"、"生铜"、"制作台"、"闪长岩"、"安山岩"、"熔炉"、火炬"、"云杉树枝"、"花岗岩"、"铁镐"、"石镐"、"木镐"、"生铁"、"木棍"、"云杉原木"、"泥土"、"鹅卵石"；
- **试验 3:** 木铲"、"木镐"、"铁砧板"、"石头"、"鳕鱼"、"煤炭"、"橡木原木"、"燧石"、"生铜"、"制作台"、"闪长岩"、"熔炉"、"安山岩"、火炬"、"花岗岩"、"青金石"、"铁镐"、"石镐"、"生铁"、"棍棒"、"砾石"、"橡木板"、"泥土"、"铁斧"、"鹅卵石"。

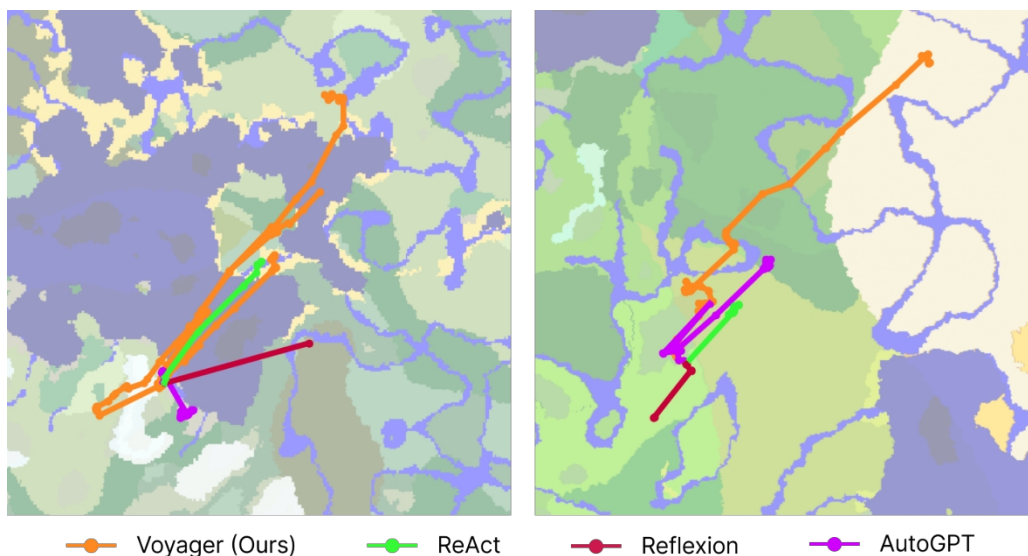


图 A.2: 地图覆盖范围: 两个 Minecraft 地图的鸟瞰图。VOYAGER 可以穿越 2.3x 与基线相比, 在穿越不同地形时的距离更长。轨迹是根据每个代理与 GPT-4 交互的位置绘制的。

B.4.2 大范围地图穿越

图 A.2。显示了代理在地图覆盖范围内的轨迹图 7 是在图 A.2 的基础上绘制的, 画出了包围每个轨迹的最小圆圈。VOYAGER 在每次试验中穿越的地形为

- 试验 1: "草地"、"沙漠"、"河流"、"热带草原"、"森林"、"平原"、"竹林"、"滴水岩洞";
- 试验 2: "雪原"、"冰冻的河流"、"滴水岩洞"、"雪山"、"海滩";
- 试验 3: "花林"、"草甸"、"古老的桦树林"、"雪坡"、"冰冻山峰"、"森林"、"河流"、"海滩"、"海洋"、"向日葵平原"、"平原"、"石质海岸"。

ReAct [29] 在每次试验中穿越的地形是

- 试验 1: "平原"、"沙漠"、"丛林";
- 试验 2: "snowy_plains" (雪原)、"snowy_taiga" (雪地)、"snowy_slopes" (雪坡);
- 试验 3: "黑暗森林"、"滴石洞穴"、"小树林"、"嶙峋山峰"。Reflexion [30]

在每个试验中穿越的地形是

- 试验 1: "平原"、"花林";
- 试验 2: "snowy_taiga";
- 试验 3: "原始森林"、"河流"、"海洋"、"海滩"、"平原"。AutoGPT [28] 在每个

试验中穿越的地形是

- 试验 1: "平原"、"滴石洞穴"、"稀树草原"、"草甸";
- 试验 2: "snowy_taiga";
- 试验 3: "平原"、"石滩"、"森林"、"海洋"。

B.4.3 对未知任务进行高效的零点归纳

图 A.3。显示了其他两项任务的零点泛化到未见任务的结果与图 8，类似VOYAGER 始终如一地解决了所有任务，而基线则无法解决任何任务。

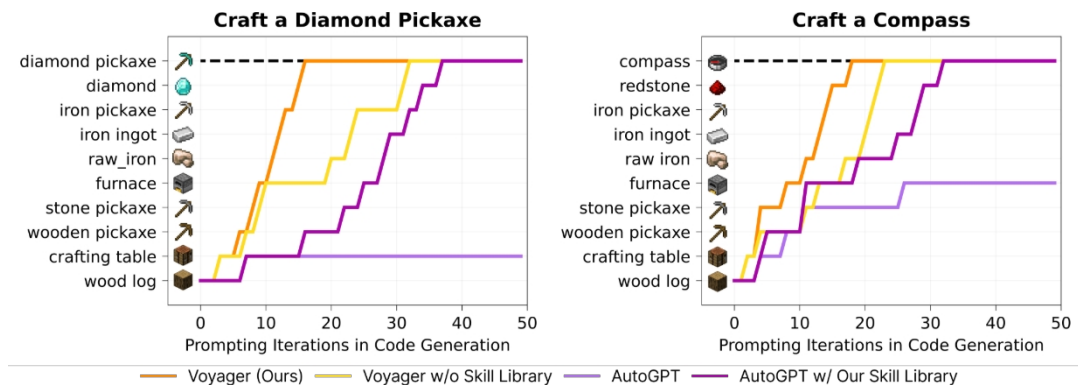


图 A.3: 对未见任务的零点泛化。我们直观地展示了每种方法在其他两个任务上的中间进展。我们没有绘制 ReAct 和 Reflexion, 因为它们没有取得任何有意义的进展。

在 50 次的提示迭代中, 我们发现了 "ReAct" 方法在任务中的进展。我们通过终身学习构建的技能库不仅提高了 VOYAGER 的性能, 还为 AutoGPT [28] 提供了助力。

B.4.4 准确的技能检索

我们对技能检索 (共 309 个样本) 进行了评估, 结果见表 A.4。A.4.前五名的准确率为 96.5%, 这表明我们的检索过程是可靠的 (注意, 我们在合成新技能的提示中包含了前五名的相关技能)。

表 A.4: 技能检索准确率。				
前 1 项 Acc	前 2 项 Acc	前 3 项 Acc	顶部-4 Acc	前 5 名 Acc
80.2± 3.0	89.3± 1.8	93.2± 0.7	95.2± 1.8	96.5± 0.3

B.4.5 模型变化的鲁棒性

在主要论文中, 旅行者号的所有实验都是使用 gpt-4-0314 进行的。我们还使用 gpt-4-0613 进行了新的实验, 发现性能大致相同 (图 A.4)。这表明 Voyager 对模型变化具有鲁棒性。

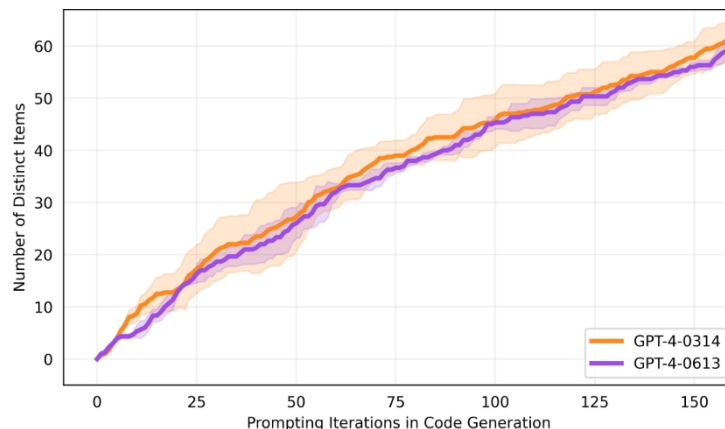


图 A.4: 使用 GPT-4-0314 和 GPT-4-0613 时 VOYAGER 的性能。